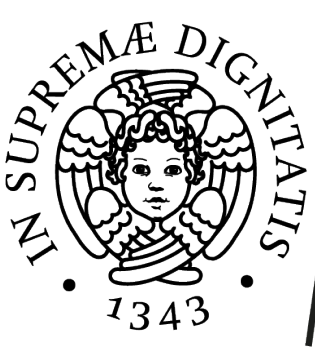

UNIVERSITÀ DI PISA

Department of Philology, Literature, and Linguistics

Master's Degree Program in Digital Humanities

GRADUATION THESIS

MAI(A) the Sound Be With You

MAIA-AV: A Ground-Up Audio-Visual Benchmark for Multimodal AI Assessment

Supervisor

Prof. Alessandro Lenci

Student

Michela Deodati

ID: 597983

Co-Supervisor(s)

Dr. Bernardo Magnini

PhD Student Davide Testa

Academic Year 2025/2026

Contents

List of Figures

List of Tables

Introduction

Determining whether the strong performance of Visual Language Models (VLMs) reflects genuine multimodal understanding, rather than the exploitation of superficial correlations and linguistic biases, remains an open challenge. *MAIA: Multimodal AI Assessment* was introduced to address this question, as a native Italian benchmark for evaluating video-based reasoning in Vision-Language Models [22, 21]. MAIA investigates the extent to which VLMs coherently integrate visual and linguistic information. More specifically, it assesses whether model predictions are grounded in visual evidence rather than primarily driven by distributional regularities learned by the language component. This distinction is crucial because a model may answer a question correctly while failing to recognise the same information consistently when it is presented in a discriminative format. In such cases, the prediction may result from linguistic plausibility rather than genuine visual understanding. To address this issue, MAIA employs two aligned tasks: Visual Statement Verification and Open-Ended Visual Question Answering. Through these tasks, the benchmark evaluates not only model accuracy, but also the robustness and consistency of model behaviour across different evaluation formats. The benchmark further defines twelve fine-grained reasoning categories to identify which competencies are used by the models and in which areas linguistic shortcuts, unimodal collapse, or hallucinations emerge. A limitation of the original framework is that the dataset was designed and evaluated primarily on the basis of visual information. Real-world videos, however, are inherently audiovisual: relevant meaning may also be conveyed through dialogue, environmental sounds, music, and other acoustic events. Recent multimodal and omnimodal architectures can process visual, auditory, and textual inputs, but the availability of multiple modalities does not guarantee that they will be integrated effectively. Models may over-rely on a single channel or be negatively affected by redundant or weakly relevant information. Building on this limitation, *Lost in Transcription: Investigating the Role of Audio Information for Video-based Visual Statement Verification* [4] extends the original MAIA research framework to a more complex analysis of the interaction between visual, linguistic, and auditory information,

while preserving its competence-oriented perspective. The study considers two complementary representations of audio: raw signal and automatic transcriptions provided as textual input. By evaluating these representations both independently and in combination with video information, the analysis investigates whether audio contributes meaningfully to task performance and whether one representation has a stronger influence on model predictions. This leads to two research questions:

RQ1: *To what extent does raw audio affect video-based Visual Statement Verification when it is jointly processed with visual and textual information?*

RQ2: *How effectively do multimodal models integrate visual, textual, and auditory information when performing video-based reasoning and understanding tasks?*

The experiments show that raw audio has a limited and strongly context-dependent effect. It is most useful when visual information is unavailable, while its addition to an already informative visual input may instead provide little benefit or introduce noise. The results also show that transcriptions can produce greater interference than raw audio, suggesting that multimodal architectures do not necessarily achieve more effective cross-modal understanding simply because additional modalities are available. These findings must nevertheless be interpreted in relation to the nature of the dataset. The one hundred original MAIA videos were selected and annotated according to their visual content, and the questions were formulated without considering the audio track. Consequently, the benchmark does not systematically require auditory information to solve its items. My thesis project begins from this limitation. The following chapters investigate whether models can integrate multiple input modalities more coherently when the dataset is specifically designed so that auditory information is at least partially relevant. The central hypothesis is that, under these conditions, models may use complementary signals across modalities to reconstruct missing information and develop a more genuinely multimodal understanding of video content, producing answers grounded in the available evidence rather than in linguistic plausibility or chance.

1. Literature review

The rapid evolution of Visual Language Models (VLMs) has progressively expanded the ability of neural systems to jointly process perceptual and linguistic information. This line of research builds on the hypothesis that linguistic meaning cannot be constructed exclusively from distributional regularities learned from textual corpora, but must also be grounded in perceptual experience [2]. Within this context, Visual Question Answering (VQA) emerged as one of the first systematic paradigms for assessing whether a model can answer questions about the content of an image [1]. Subsequent benchmarks, such as GQA, extended this setting by introducing compositional questions involving entities, attributes, and visual relations [9]. However, high task accuracy alone does not provide sufficient evidence of multimodal comprehension. Several studies have shown that models can exploit statistical regularities in answers, questions, or dataset distributions, producing correct predictions without necessarily *understanding* the associated visual content. Evaluation has therefore progressively moved beyond task performance towards the assessment of grounding, understood as the ability to associate a prediction with the relevant perceptual evidence. One widely adopted strategy for evaluating grounding relies on minimally contrastive pairs, in which a correct instance is paired with a foil obtained by modifying a semantically relevant element. *FOIL-it!*, for example, evaluates whether a model can detect localized inconsistencies between an image and its linguistic description [19]. *Winoground*, instead, presents pairs of images and descriptions containing the same lexical elements but arranged according to different compositional relations, showing that recognizing individual entities does not necessarily entail understanding the relational structure connecting them [23]. The transition from images to videos introduces additional challenges. Video understanding requires integrating information distributed over time, recognizing actions and state changes, ordering events, and establishing temporal and causal relations. *Action Genome Question Answering (AGQA)* addresses these requirements through compositional questions concerning objects, actions, and spatio-temporal relations [8]. The *Multi-modal Video Understanding Benchmark (MVBench)*

similarly evaluates a broader range of competencies, including action perception, movement, temporal ordering, and dynamic reasoning [13]. Nevertheless, video benchmarks remain vulnerable to unimodal shortcuts and to strategies that rely predominantly on the most informative frame. *Video Language Model Assessment (VILMA)* addresses this limitation as a zero-shot benchmark based on video–text contrastive pairs [10]. For each complex capability, VILMA introduces a proficiency test designed to verify the preliminary perceptual ability required to solve the corresponding task. The results show that, once these prerequisite conditions are considered, part of the apparently correct performance can be attributed to accidental or spurious strategies. In particular, the examined VLMs do not display a systematic advantage over statistical image-based models on tasks requiring strict temporal grounding. Within this line of research, MAIA introduced a natively Italian benchmark designed to provide a fine-grained evaluation of video-linguistic reasoning [22]. The resource covers twelve reasoning categories related to linguistic phenomena, including causal, counterfactual, implicit, spatial, temporal, sentimental, planning, uncertainty, and out-of-scope reasoning. Its main contribution lies in aligning two tasks built on the same videos and questions: *Visual Statement Verification (VSV)*, where the model selects the correct statement from a minimally contrastive pair, and *Open-Ended Visual Question Answering (OEVQA)*, where the model freely generates the answer. This setting makes it possible to assess not only accuracy but also coherence between discriminative comprehension and linguistic generation. A model may correctly answer a multiple-choice question while failing to reproduce the same information in an open-ended setting, or may behave inconsistently across equivalent linguistic formulations. To capture this phenomenon, MAIA introduces the *Aggregate Accuracy* metric, which rewards only those instances in which the model correctly answers all true–false pairs associated with the same question and simultaneously generates the correct response in the corresponding open-ended task. The reduction in Aggregate Accuracy relative to conventional single-instance accuracy highlights the fragility of apparently strong performance and the need to jointly evaluate robustness, consistency, and grounding. Although VLMs have traditionally focused on text–vision interactions, real videos are intrinsically audiovisual. Their interpretation may depend on music, prosody, background noise, and other acoustic signals

that cannot be reconstructed from visual frames alone. An increasing body of research has accordingly extended multimodal language model architectures to support joint processing of text, images, video, and audio. *ImageBind*, for instance, introduced a shared representation space spanning images, text, audio, depth, thermal information, and inertial movement [7]. Other multimodal language models (MLMs) rely on modality-specific encoders combined with projection or fusion modules that align heterogeneous representations with the linguistic space. *Video-SALMONN: Speech-Enhanced Audio-Visual Large Language Models* introduces a multi-resolution Q-former[1] to align audio signals with synchronized video [20], while *VideoLLaMA2* combines spatio-temporal modeling and acoustic processing within a generative multimodal architecture [3]. More recent omnimodal architectures process multiple modalities within a unified framework. *Qwen2.5-Omni* adopts a thinker–talker architecture that accepts text, image, video, and audio inputs and produces textual or audio responses [14]. The subsequent *Qwen3.5-Omni* architecture integrates visual and audio encoders into a common backbone and introduces explicit temporal references to improve audio–video alignment [15]. These developments demonstrate that heterogeneous signals can be processed within a single architecture. The availability of a modality, however, does not imply that it is used functionally or complementarily. The distinction between **multimodal availability** and **multimodal integration** is essential here: the former indicates that a system can accept multiple input sources, the latter requires the model to select, coordinate, and combine relevant information across modalities. A system may thus be multimodal from an architectural perspective while relying predominantly on unimodal information during prediction. This issue is evident in *Audio-Visual Question Answering* benchmarks such as MUSIC-AVQA, which evaluates spatio-temporal relations in musical performance videos and extends the analysis to objects, entities, and everyday activities [12]. Subsequent studies reveal a strong visual bias, showing that many questions can be answered using the video stream alone, without exploiting the audio channel. Under these conditions, similar audio-only and video-only performance does not demon-

[1] A multi-resolution Q-former, such as a Multi-Resolution Causal Q-Former, processes continuous audio, video, or image information at different temporal or spatial scales through distinct query banks, balancing fine-grained signals with broader contextual information.

strate genuine integration, but rather suggests a form of unimodal collapse in which one modality dominates the decision process. To address this problem, *Dave* was introduced as a benchmark in which each question jointly requires visual and acoustic information, making either modality insufficient in isolation [16]. The benchmark distinguishes among action recognition, sound classification, temporal ordering, and multimodal synchronization, a decomposition that helps determine whether an error arises from the failure to perceive an event or from the inability to establish temporal relations between sounds and visual actions. The results show that models may achieve strong performance on individual components while encountering greater difficulty in audio–visual synchronization, indicating that cross-modal integration remains a central limitation. The increasing attention to audio has also led to benchmarks specifically designed to assess acoustic intelligence. *MMAU-Pro* evaluates 49 competencies distributed across speech, environmental sounds, music, and their combinations [11]. These tasks include long-audio comprehension, multi-track reasoning, spatial localization, prosody interpretation, and the integration of dialogue, music, and background noise. The persistently limited performance of current models indicates that acoustic processing cannot be reduced to dialogue recognition alone, since audio also conveys spatial, temporal, emotional, and contextual information that must be evaluated independently. A complementary perspective is provided by *SONIC-01*, a benchmark for omnimodal models based on audio–visual interactions drawn from real conversational and everyday contexts [17]. SONIC-01 combines open-ended summarization, multiple-choice questions, and temporal event localization, while also showing that replacing raw audio with transcription removes paralinguistic information related to tone and communicative intention. Its results identify temporal localization as one of the most challenging tasks, confirming that the integration of verbal content, acoustic signals, and visual sequences remains an open problem. The comparison between native audio and transcription is also central to my recent work, *Lost in Translation: Investigating the Role of Audio Information for Video-based Visual Statement Verification* [4]. The study extends the MAIA framework by introducing audio into the VSV task and comparing two representations of the acoustic modality: raw audio directly processed by *Qwen2.5-Omni-7B* and automatic transcriptions supplied as additional textual input to *Qwen2.5-VL-7B*. To characterize the

audio content, the videos are divided into four classes:

Music: Instances characterized by repetitive vocalizations, onomatopoeic patterns, strongly repetitive tokens, or explicit lexical cues associated with non-speech audio.

Dialogue: Videos whose transcriptions display sufficiently coherent linguistic content, acceptable log-probability values, low degeneracy, and limited evidence of no-speech segments.

Silence/Noise: Videos with empty or highly unreliable transcriptions, particularly when associated with low log-probability values and relatively high no-speech probabilities.

Unknown: Residual cases that appear speech-related but do not satisfy the dialogue criteria with sufficient confidence, thereby preserving potentially informative content while explicitly marking its uncertainty.

Class labels and speech transcriptions were obtained by combining the outputs of *Whisper-1* and *GPT-4o-transcribe*, together with an assessment of output plausibility. This classification distinguishes results according to the informativeness of the available audio content. The results reveal a marked dominance of the visual channel. Audio provides only a limited benefit, mainly when no visual information is available. Once rich visual input is provided, audio's contribution becomes negligible and may even introduce noise or errors, with limited robustness observed when a single black frame is added instead. Under visually rich conditions, the difference between with-audio and without-audio settings is minimal, suggesting that audio functions more as a surrogate channel than as information fully integrated with vision. Audio transcriptions exhibit a different behavior. When added to already rich visual input, they reduce accuracy relative to the visual-only condition. This shows that transcription is not a neutral transformation: once projected into the same representational space as the questions and answer options, it can compete with them, increasing the influence of textual regularities and introducing redundant or irrelevant information. *Lost in Translation* thus shows that different representations of the same acoustic content can affect the model's decision process differently. Raw audio can be

partially ignored when it is not useful, whereas transcription interacts directly with the linguistic component of the model. Introducing an additional modality does not therefore guarantee an improvement, and may instead compromise predictions that were already correct on the basis of reliable visual evidence. These findings must be interpreted in light of the original MAIA design. The benchmark was created to investigate visual content, and annotators were explicitly instructed not to consider audio information, so its questions and answers were not designed to exploit acoustic evidence. The limited contribution of audio may therefore reflect both the models' modality-integration mechanisms and the limited relevance of audio within the original dataset. Recent literature further emphasizes the importance of identifying the distinct sources of multimodal errors. *MIRAGE* disentangles hallucinations caused by perceptual failures from those arising from incorrect reasoning over correctly represented information [5]. Although primarily focused on visual reasoning, its methodological principle extends naturally to audio–visual settings, where an incorrect response may derive from failed sound recognition, inaccurate temporal synchronization, selection of the wrong modality, or erroneous inference over otherwise correctly represented evidence. Complementary attribution approaches seek to identify which parts of the input contribute to the generation of an answer. *OmniTrace* proposes an attribution framework for omnimodal autoregressive generation that links generated tokens to textual segments, visual regions, and audio/video intervals, aggregating attention signals into cross-modal explanations associated with linguistically interpretable units. This type of analysis is particularly relevant for determining whether a model actually exploits audio information or whether its prediction is driven primarily by text and vision. The current state of the art thus reveals a clear tension between the rapid development of omnimodal architectures and their effective ability to integrate heterogeneous modalities. Contemporary models can receive multiple input types within the same context, yet diagnostic benchmarks provide no guarantee that these sources are used complementarily. Performance may instead depend on a dominant modality, linguistic shortcuts, dataset correlations, or additional signals that introduce noise. A central unresolved limitation is the construction of benchmarks in which audio and visual information are explicitly supervised. Assessing genuine audio–visual comprehension requires items in which audio contributes relevant

information and the correct answer cannot be determined from a single modality alone. Such a design would distinguish complementarity, dominance, substitution, and interference, while evaluating whether the model selects the appropriate evidence and combines it coherently across modalities. The present study follows this direction. Building on the competence-oriented MAIA framework and the recent findings of *Lost in Translation*, it investigates model behavior on a dataset specifically designed to require a genuine contribution from audio in order to answer the questions correctly. The central hypothesis is that, when audio and video provide partial and complementary evidence, multimodal comprehension depends on the model's ability to reconstruct the missing information through cross-modal integration, avoiding both unimodal collapse and linguistic shortcuts based on plausibility.

2. Data Collection

2.1 From MAIA to MAIA-AV: Dataset Expansion and Curation

Video Selection

The dataset adopted in MAIA-AV builds upon the guidelines of Multimodal AI Assessment (MAIA), comprising 100 short videos, approximately 30 seconds each, selected from YouTube Italia to represent everyday scenarios of Italian culture. Content domains include sports, culinary arts, urban life, and cultural activities, with a strong focus on human presence and highly informative scenes. In the original formulation, video selection and annotation were oriented strictly toward visual comprehension, completely omitting audio content. Building upon this initial set, we collected an additional 100 videos, each uniquely identified from `video001` to `video100`. These identifiers map each video to its corresponding audio stream and text transcriptions. Each video was precisely trimmed to extract targeted segments, creating a controlled corpus of clips. Particular attention was paid to the alignment between dialogue content and the elements appearing in the scene. This targeted selection of relevant dialogue enables a precise investigation of multimodal interactions. By prioritizing scenarios where spoken dialogue is semantically grounded in the visual content, we ensure that the audio signal plays a genuinely informative role in the multimodal instance. This distinction represents the core departure from the original dataset: MAIA-AV is a benchmark engineered to combine and compare distributed evidence across visual, auditory, and textual modalities.

Implementation Details The video acquisition pipeline relies on `yt_dlp` to retrieve targeted temporal segments from selected YouTube videos. Rather than downloading complete sources for post-hoc editing, each clip is specified prior to acquisition via three parameters: source URL, start timestamp, and duration. These parameters configure a pre-

cise temporal interval directly governing the retrieval process. Timestamp conversion into seconds is handled by `parse_time`, which reduces `HH:MM:SS` or `MM:SS` strings into integer values:

```
def parse_time(time_str):
    """Converts HH:MM:SS or MM:SS string to seconds."""
    parts = [int(p) for p in time_str.split(':')]

    if len(parts) == 3:
        return parts[0] * 3600 + parts[1] * 60 + parts[2]
    elif len(parts) == 2:
        return parts[0] * 60 + parts[1]
    return parts[0]
```

Adding the requested duration to the start position determines the upper boundary of the segment:

```
start_sec = parse_time(start_time_str)
end_sec = start_sec + int(durata_str)
```

The resulting bounds, `start_sec` and `end_sec`, define the exact temporal window extracted from the source. Output paths are structured prior to downloader configuration, ensuring MP4 container compliance and consistent dataset indexing under the target directory:

```
filename_input = input(
    "Enter output filename (e.g., clip): "
).strip()
if not filename_input.endswith(".mp4"):
    filename_input += ".mp4"
output = f"Dataset/video{filename_input}"
```

Prefixing filenames with `video` maintains a unified identifier convention across all dataset instances. Stream selection, temporal trimming, and media normalization are encapsulated within the `ydl_opts` dictionary. Format selection prioritizes high-resolution video streams

alongside optimal audio feeds:

```
ydl_opts = {
    'format': 'bestvideo[height<=2160]+bestaudio/best',
}
```

This filter selects video streams up to 2160p paired with the highest quality audio stream, falling back to pre-merged streams (`/best`) if separate tracks are unavailable. Temporal constraints are applied during acquisition via `download_range_func`:

```
    'download_ranges': download_range_func(
        None,
        [(start_sec, end_sec)]
    ),
```

Embedding segment bounds directly into the retrieval mechanism eliminates separate media cropping steps and reduces bandwidth overhead. To maintain frame-accurate cutting across keyframe-based compressed video formats, the pipeline enforces keyframe generation at boundary points:

```
    'force_keyframes_at_cuts': True,
```

Forcing keyframes ensures temporal alignment at cut points, preserving action continuity for downstream multimodal evaluation in MAIA-AV. Output formats and post-processing codecs are explicitly declared to standardize the corpus:

```
    'merge_output_format': 'mp4',
    'postprocessor_args': {
        'ffmpeg': [
            '-c:v', 'libx264',
            '-preset', 'fast',
            '-c:a', 'aac'
        ]
    },
```

FFmpeg re-encodes visual streams via H.264 (`libx264`, `fast` preset) and acoustic streams

via AAC, eliminating encoding heterogeneity across raw YouTube sources. The unified configuration dictionary combines acquisition parameters, formatting rules, output paths, and authentication credentials:

```
ydl_opts = {
    'format': 'bestvideo[height<=2160]+bestaudio/best',
    'download_ranges': download_range_func(
        None,
        [(start_sec, end_sec)]
    ),
    'force_keyframes_at_cuts': True,
    'merge_output_format': 'mp4',
    'postprocessor_args': {
        'ffmpeg': [
            '-c:v', 'libx264',
            '-preset', 'fast',
            '-c:a', 'aac'
        ]
    },
    'outtmpl': output,
    'cookiefile': 'src/youtube.com_cookies.txt',
}
```

Passing a session cookie file (`cookiefile`) enables retrieval of age-restricted or session-gated content without altering core extraction logic. Execution is wrapped in exception handling blocks, logging errors and returning non-zero exit codes upon failure:

```
try:
    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        ydl.download([url])
    print(f"\nSuccessfully downloaded clip to {output}")
except Exception as e:
    print(f"\nError: {str(e)}")
    sys.exit(1)
```

This setup allows `yt_dlp` to coordinate format resolution, stream extraction, interval clipping, and FFmpeg transcoding within a single execution pass.

Audio Conversion

In the MAIA-AV framework, the auditory element of each video is regarded as an autonomous source of information rather than a mere supplementary part of the visual narrative. Spoken language, ambient sounds, and acoustic phenomena offer significant evidence that enhances visual data and aids in scene interpretation. Isolating the audio allows this modality to be independently analyzed, ensuring its contribution is maintained throughout subsequent processing stages. For every video, the original audio track is extracted and archived as a separate file. This separation retains the identical temporal parameters and identifier associated with the corresponding audiovisual entity. Such a one-to-one mapping upholds alignment across modalities, enabling visual, acoustic, and textual representations to be traced back to the same foundational event. The resulting audio files function as direct inputs for further pipeline stages, including automatic speech transcription and specialized audio analyses. The extraction process neither generates a modified version of the original content nor alters it; instead, it separates the auditory channel, allowing independent processing while remaining completely synchronized with the originating video content.

Implementation Details The audio extraction stage converts each audiovisual dataset instance into a standalone acoustic representation while preserving multimodal alignment with the source video. Implemented via Python's `subprocess` module, the pipeline invokes FFmpeg to automate batch processing across the complete video collection. Input and output paths are declared explicitly at execution setup:

```
INPUT_FOLDER = Path("data/video")
OUTPUT_FOLDER = Path("data/audio")
VIDEO_EXTENSIONS = {".mp4"}
```

Decoupling input and output directories isolates raw video resources and establishes an aligned audio mirror directory. Restricting `VIDEO_EXTENSIONS` to MP4 matches the output format produced by the preceding acquisition stage. The core conversion routine is encapsulated within `mp4_to_mp3`:

```
def mp4_to_mp3(
    input_path: Union[str, Path],
    output_path: Optional[Union[str, Path]] = None,
    bitrate: str = "192k",
) -> Path:
    """Extract the audio from an MP4 file and save it as MP3."""
```

When an explicit destination path is omitted, the function retains the base filename and replaces the container extension:

```
    input_path = Path(input_path)
    output_path = (
        input_path.with_suffix(".mp3")
        if output_path is None
        else Path(output_path)
    )
```

This mapping strategy guarantees cross-modal alignment across representations (e.g., `video001.mp4` yields `video001.mp3`). Prior to invoking FFmpeg, parent directory existence is enforced:

```
    output_path.parent.mkdir(
        parents=True,
        exist_ok=True,
    )
```

This step allows nested input folder hierarchies to be automatically replicated within the output directory. Extraction is executed using a single `subprocess.run` call:

```
    subprocess.run(
        [
            "ffmpeg",
```

```
            "-y",
            "-i",
            str(input_path),
            "-vn",
            "-c:a",
            "libmp3lame",
            "-b:a",
            bitrate,
            str(output_path),
        ],
        check=True,
    )
```

The `-vn` flag discards video streams, ensuring that FFmpeg isolates the acoustic signal rather than performing generic media transcode operations. Acoustic streams are normalized using the `libmp3lame` codec at a constant target bitrate of 192 kbit/s (`-b:a 192k`). Passing `-y` overwrites pre-existing destination files without prompt interruption, facilitating seamless pipeline re-execution. Setting `check=True` ensures runtime errors raise a `CalledProcessError` rather than failing silently. Upon completion, the function returns the confirmed target path:

```
    return output_path
```

Batch processing begins by recursively indexing and sorting compatible media files:

```
def find_videos(folder: Path) -> list[Path]:
    """Return all supported video files found recursively."""
    return sorted(
        path
        for path in folder.rglob("*")
        if path.is_file()
        and path.suffix.lower() in VIDEO_EXTENSIONS
    )
```

Sorting file paths guarantees deterministic execution order across operating environments. Directory validity is verified before initializing conversion loops:

```
if not INPUT_FOLDER.exists():
    raise FileNotFoundError(
        f"Input folder not found: {INPUT_FOLDER}"
    )
videos = find_videos(INPUT_FOLDER)
if not videos:
    print(f"No videos found in: {INPUT_FOLDER}")
    return
```

Validating the input directory early avoids redundant FFmpeg invocations when processing empty or invalid paths. For each video, relative path resolution preserves source subdirectory structures under `OUTPUT_FOLDER`:

```
for index, video_path in enumerate(videos, start=1):
    relative_path = video_path.relative_to(INPUT_FOLDER)
    audio_path = (
        OUTPUT_FOLDER
        / relative_path.with_suffix(".mp3")
    )
```

Replicating source directory trees ensures structural parity between visual and acoustic datasets. Each file conversion is executed inside localized exception blocks:

```
    try:
        mp4_to_mp3(
            video_path,
            audio_path,
        )
        print("Audio extraction completed.")
    except subprocess.CalledProcessError as error:
        print(f"FFmpeg error: {error}")
```

Capturing `CalledProcessError` exceptions locally prevents single-file execution failures from interrupting broader batch operations while logging specific processing errors.

Transcription Extraction

Another key representation in our dataset is the audio transcription, following the methodology established in our previous work [4]. Transcriptions were generated using the *Faster-Whisper* model (`large-v3`), a reimplementation of *OpenAI's Whisper* powered by the *CTranslate2* fast inference engine for Transformer architectures. The model processes raw audio content directly, with the language parameter set to Italian. Additionally, a *Voice Activity Detection* (VAD) filter is applied to isolate segments containing actual speech, thereby minimizing non-speech transcriptions. The overall transcription procedure aims to convert spoken dialogue into textual content that can be reliably recognized and processed by downstream models. This methodology represents an evolution compared to previous approaches on acoustic content analysis [4], which jointly employed *Whisper-1* and *GPT-4o-transcribe* to compare and align outputs for higher precision. In contrast, the current pipeline streamlines the transcription workflow by deploying a single, locally executed model, providing an efficient and reproducible framework.

Implementation Details The transcription stage was implemented as an independent component of the data preparation pipeline to associate each video with a textual representation of its spoken content. Powered by `Faster-Whisper` and the `large-v3` model executed locally on GPU, this setup ensures full reproducibility while eliminating dependencies on external cloud services. The execution environment explicitly configures a selected CUDA device and establishes a local cache for model files:

```
    os.environ["CUDA_VISIBLE_DEVICES"] = "0"

    HF_CACHE = Path.cwd() / ".hf_cache"
    HF_CACHE.mkdir(parents=True, exist_ok=True)

    os.environ["HF_HOME"] = str(HF_CACHE)
```

```
    os.environ["HF_HUB_CACHE"] = str(HF_CACHE / "hub")
```

Utilizing a local cache confines model weights to the working directory, preventing redundant downloads across system locations. The transcription model is subsequently instantiated on CUDA using `float16` precision:

```
    MODEL_NAME = "large-v3"
    LANGUAGE = "it"

    model = WhisperModel(
        MODEL_NAME,
        device="cuda",
        device_index=0,
        compute_type="float16",
        download_root=str(HF_CACHE / "hub"),
    )
```

Setting `language="it"` explicitly conditions decoding on Italian. This parameter bypasses automatic language identification and directly aligns model execution with the linguistic profile of the dataset. Prior to processing, the pipeline recursively retrieves and filters all `.mp4` files within the input directory, sorting the resulting collection to guarantee deterministic execution order:

```
    def find_videos(folder: Path) -> list[Path]:
        return sorted(
            path
            for path in folder.rglob("*")
            if path.is_file()
            and path.suffix.lower() in VIDEO_EXTENSIONS
        )
```

The core transcription workflow relies on `model.transcribe()`. By directly parsing the audio embedded within the video container, Faster-Whisper eliminates intermediate extraction steps and yields chronologically ordered transcription segments:

```
    segments, _ = model.transcribe(
        str(video_path),
        language=LANGUAGE,
        beam_size=5,
        vad_filter=True,
        condition_on_previous_text=False,
    )
```

Three key hyperparameters govern decoding precision and stability:

- `beam_size=5`: Activates beam-search decoding, evaluating multiple competing hypotheses during generation to balance accuracy and computational overhead.
- `vad_filter=True`: Enables Voice Activity Detection to strip environmental noise, music, and unvoiced pauses, preventing non-linguistic signals from reaching the ASR engine.
- `condition_on_previous_text=False`: Disables cross-segment context conditioning, isolating individual speech segments and mitigating error propagation across the audio stream.

The resulting segment iterator is aggregated into a continuous transcript by stripping whitespace, discarding empty segments, and concatenating valid text chronologically:

```
    return " ".join(
        segment.text.strip()
        for segment in segments
        if segment.text.strip()
    )
```

Omitting downstream linguistic post-processing or semantic filtering preserves an unadulterated representation of raw ASR performance. The corpus is processed sequentially, using each video filename as a unique primary key:

```
    transcriptions = {}
```

```
    for index, video_path in enumerate(videos, start=1):
    try:
        transcription = transcribe_video(video_path)
        transcriptions[video_path.name] = transcription
    except Exception as error:
        transcriptions[video_path.name] = (
            f"ERROR: {error}"
        )
```

Robust error handling isolates processing failures on individual files, preventing execution halts and cleanly decoupling unvoiced media from runtime exceptions. Finally, extracted transcripts are exported to a structured CSV file indexed by `video_name` and `transcription`:

```
    writer = csv.DictWriter(
        csv_file,
        fieldnames=[
        "video_name",
        "transcription"
        ],
        delimiter=";",
    )
    writer.writeheader()
    writer.writerows(
        {
            "video_name": video_name,
            "transcription": transcription,
        }
        for video_name, transcription
        in transcriptions.items()
    )
```

2.2 Expanding the Questions and Answers Dataset

The construction of linguistic data in MAIA-AV mirrors the methodological framework established in the original MAIA benchmark. The process retains the question as the foundational element for subsequent answer collection and the creation of an evaluation dataset. In the original MAIA benchmark, each video was associated with 2,400 questions. Expert annotators, operating under specific category guidelines, were responsible for generating these questions. The questions were designed in an open-ended format to preclude simple binary responses, such as yes or no. Given that the original benchmark was intended for models based solely on visual input, annotators were explicitly instructed to disregard all acoustic data during the question formulation phase. MAIA-AV extends the original framework by modifying the informational basis for question generation. This version includes a total of 300 questions, with each video offering three inquiries that delve into spatial, temporal, and causal dimensions of the presented information. These categories are aligned with the structure of the original MAIA benchmark. The spatial category focuses on identifying locations or entities, either visible or referenced, along with detailing the spatial relationships between them. The temporal category examines event sequences, identifying when specific occurrences take place and establishing temporal relationships such as preceding, succeeding, or simultaneous actions. The causal category investigates the reasons behind or consequences of actions, events, or behaviors observed within the video context. The primary distinction between the original MAIA and its successor, MAIA-AV, is the integration of audio information, which is crucial in the latter version. Certain questions are specifically designed to address audio elements of the video, including character dialogue, vocal commands to infotainment systems, and ambient sounds. This addition incorporates linguistic content that connects information across both visual and auditory channels. The process of acquiring answers adheres to the MAIA guidelines. For the newly devised set of 300 questions, data was gathered through a questionnaire distributed among four volunteers. These individuals are native speakers of Italian, each born in Italy, and possessing at least a high school diploma. These participants engaged with 100 videos, responding to the corresponding 300 questions, thus

enabling the collection of independent responses and four distinct articulations of each answer. The structured approach remains intact: the videos operate as the multimodal information source, the questions identify the specific information to be extracted, and the human-generated answers provide the necessary linguistic reference for constructing subsequent components of the benchmark.

Caption-Foil Generation

Building upon the human-annotated responses, the construction of *MAIA-AV* proceeds with the automated generation of *caption–foil* pairs for the Visual Statement Verification (VSV) task. This procedure follows the methodology established in *MAIA*: each question-answer pair is first converted into a declarative true statement (*True Statement*, TS) and subsequently modified to create a minimally contrastive false statement (*False Statement*, FS). Consequently, the caption and foil share maximum linguistic structure, differing primarily in question-relevant information. While the original benchmark collects eight human responses per question to yield eight TS–FS pairs, *MAIA-AV* adopts a scaled-down approach: four independent responses per question yield four captions and four corresponding foils. With three questions per video (spatial, temporal, and causal), each video comprises twelve caption–foil pairs. The automated generation employs `GPT-4o-2024-08-06`, matching the exact model version used in the original *MAIA* pipeline. Generation proceeds in two distinct stages. In the initial stage, the model receives only the question and a single human answer. Rather than generating novel content, the prompt directs the model to synthesize the question–answer pair into a single declarative statement preserving the original semantic content. The instructions mandate minimal alterations to annotators' phrasing while requiring the conversion of first-person expressions into an impersonal stance. Accordingly, phrases like "I think that" or "I believe that" are removed from the caption without discarding any underlying uncertainty. Restricting outputs to concise declarative sentences leverages natural annotator variance while avoiding reliance on the model for factual core generation. This initial phase employs the prompt used for True Statement generation in *MAIA*:

```
Sei un assistente che crea affermazioni in italiano sui video.
```

```
Data una domanda Q e una risposta A relative a un video, devi creare
un'affermazione S basata su A.
Mentre generi S, cerca di non alterare le parole che compongono A.
Se A include verbi o frasi in prima persona (ad es., 'penso', 'credo'),
riformula S in modo impersonale, evitando la prima persona.
L'affermazione deve essere una frase breve e dichiarativa.[1]
```

The instructions include few-shot examples demonstrating question–answer fusion, with specific coverage of negative responses, non-deducible information, and expressions of uncertainty. Appended to the prompt, the target question and annotator response appear in `Q:` and `A:` format, conditioning the model to output solely the statement `S:`. The second stage processes the generated caption to yield the corresponding foil. Unlike the initial phase, this prompt adapts to the target question category. The core methodology relies on constructing *minimal pairs*: the model modifies only the information necessary to invalidate the caption, avoiding extraneous additions and preserving sentence structure. Consequently, distinguishing between caption and foil tests a model's ability to ground claims in audiovisual content rather than relying on surface linguistic cues. This design adheres to *MAIA*, directing GPT-4o to alter exclusively elements relevant to the specified semantic domain. Across all categories, the system prompt assigns a uniform role:

```
Sei un assistente progettato per creare foil a partire dalle caption.[2]
```

Subsequent instructions vary by targeted semantic dimension. For **spatial** questions, foil generation alters strictly the entity's position or location:

```
Data una caption in italiano (C) riguardante la posizione o la localizzazione
di qualcuno o qualcosa, il tuo compito è creare il suo foil (F) modificando
solo l'informazione spaziale.
```

[1]English translation: *You are an assistant that creates statements in Italian about videos. Given a question Q and an answer A concerning a video, you must create a statement S based on A. While generating S, try not to alter the words composing A. If A includes first-person verbs or phrases (e.g., 'I think,' 'I believe'), rephrase S to be impersonal, avoiding a first-person perspective. The statement should be a concise, declarative sentence.*

[2]English translation: *You are an assistant designed to create foils based on captions.*

`Non aggiungere altre informazioni rispetto a quanto dichiarato in C.`[3]

An accompanying example relocates a woman standing in a poppy field to a school setting. Preserving the subject and surrounding text isolates spatial information as the sole determinant of truth value. For **temporal** questions, instructions dictate altering the temporal relationships within the caption:

`Data una caption in italiano (C) riguardante gli eventi e quando avvengono, il tuo compito è creare il suo foil (F) modificando solo l'informazione temporale.`
`Non aggiungere altre informazioni rispetto a quanto dichiarato in C.`[4]

An illustrative example converts the chronological order of two events into an alternative sequence, preserving core semantic content while modifying event timing or order. For **causal** questions, foil construction alters the cause-and-effect relationship expressed in the caption:

`Data una caption in italiano (C) riguardante le cause o gli effetti degli eventi, il tuo compito è creare il suo foil (F) modificando le cause o l'effetto dell'evento principale.`
`Non aggiungere altre informazioni rispetto a quanto dichiarato in C.`[5]

The provided example retains the principal action, such as a person beginning to run, while altering solely the underlying cause. This focuses the contrast on the causal link

[3]English translation: *Given an Italian caption (C) regarding the position or location of someone or something, your task is to create its foil (F) by changing only the spatial information. Don't add other information with respect to what is stated in C.*

[4]English translation: *Given an Italian caption (C) regarding events and when they happen, your task is to create its foil (F) by changing only the temporal information. Don't add other information with respect to what is stated in C.*

[5]English translation: *Given an Italian caption (C) regarding the causes or the effects of events, your task is to create its foil (F) by changing the causes or the effect of the main event. Don't add other information with respect to what is stated in C.*

rather than arbitrary scene modifications. In the final task setup, option ordering is randomized, preventing positional bias and ensuring performance metrics reflect semantic comprehension rather than structural pattern exploitation.

Implementation Details The *caption–foil* generation pipeline is implemented in `caption_foil.py`. The script relies on `pandas` for tabular data processing and the OpenAI Python client for model inference. Both caption and foil generation utilize `gpt-4o-2024-08-06`, with authentication managed via the `OPENAI_API_KEY` environment variable. Input and output paths, along with the expected annotation structure, are defined at initialization:

```
MODEL = "gpt-4o-2024-08-06"
RAW_ANSWERS_DIR = Path("data/vsv/raw_human_answers")
QUESTIONS_FILE = Path("data/vsv/question_classification.csv")
OUTPUT_DIR = Path("data/vsv/caption-foil")
CHECKPOINT_FILE = OUTPUT_DIR / "caption_foil_checkpoint.csv"
OUTPUT_FILE = OUTPUT_DIR / "caption_foil.csv"
RANDOM_SEED = 42
EXPECTED_MACROAREAS = {"spaziale", "temporale", "causale"}
EXPECTED_ANNOTATORS = 4
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
```

Questions are loaded from `data/vsv/question_classification.csv`. Only the video identifier, principal dimension, and question text are required for this stage:

```
questions = pd.read_csv(
    QUESTIONS_FILE,
    sep=";",
    encoding="utf-8-sig",
)
questions = questions[
    ["video_name", "principal_dimension", "question_text"]
].copy()
```

Prior to processing, the script verifies column availability, discards missing values, nor-

malizes category labels, and confirms the presence of only the three expected macroareas. Video identifiers are standardized using `normalize_video_name()`, which extracts the numeric identifier and converts it to the format used across the dataset:

```
def normalize_video_name(value):
    value = str(value).strip()
    match = re.search(r"(\d+)", value)
    if not match:
        raise ValueError(
            f"Impossibile ricavare il numero del video da: {value}"
        )

    return f"video{int(match.group(1)):03d}.mp4"
```

Additionally, the script ensures that each video is associated with exactly one spatial, one temporal, and one causal question. Duplicate video–macroarea pairs are rejected prior to generation. Human responses are automatically loaded from all CSV files within `data/vsv/raw_human_answers`. The file count is validated against the expected number of annotators:

```
files = sorted(RAW_ANSWERS_DIR.glob("*.csv"))
if len(files) != EXPECTED_ANNOTATORS:
    raise ValueError(
        f"Attesi {EXPECTED_ANNOTATORS} file di annotatori in "
        f"{RAW_ANSWERS_DIR}, trovati {len(files)}"
    )
```

Each file must contain the columns `titolo_video`, `macroarea`, and `risposta`. For backward compatibility, the alternative column name `risposte` is also accepted and automatically renamed. Annotator identifiers are extracted directly from the filenames:

```
if "risposta" not in df.columns and "risposte" in df.columns:
    df = df.rename(columns={"risposte": "risposta"})
```

```
df = df[["titolo_video", "macroarea", "risposta"]].copy()
df["video_name"] = df["titolo_video"].map(
    normalize_video_name
)
df["macroarea"] = (
    df["macroarea"]
    .astype(str)
    .str.lower()
    .str.strip()
)
df["annotator"] = file.stem.removeprefix("risposte_")
```

Questions and responses are subsequently aligned via the shared `video_name` and `macroarea` fields. The merge is constrained as *many-to-one*, matching multiple annotator responses to a single question:

```
data = answers.merge(
    questions,
    on=["video_name", "macroarea"],
    how="left",
    validate="many_to_one",
)
```

Additional consistency checks validate question keys against annotator responses. Missing answers or orphan responses interrupt execution prior to invoking any API requests. Responses corresponding to the same video and macroarea are grouped into a common pool. The pool identifier is formed by concatenating the video identifier with the corresponding macroarea:

```
data["video_id"] = (
    data["video_name"].str.removesuffix(".mp4")
)
data["pool_id"] = (
    data["video_id"] + "_" + data["macroarea"]
```

```
)
```

Each pool must contain exactly four responses. Individual entries are assigned a sequential position and a unique identifier:

```
data["pool_item"] = (
    data.groupby("pool_id").cumcount() + 1
)
data["id"] = (
    data["pool_id"]
    + "_"
    + data["pool_item"].astype(str).str.zfill(2)
)
```

API calls are centralized within `call_openai()`, which serves both caption and foil generation functions:

```
response = client.chat.completions.create(
    model=MODEL,
    messages=messages,
    max_tokens=max_tokens,
)
return response.choices[0].message.content.strip()
```

The function executes up to five attempts for recoverable API failures. Rate-limit errors trigger a 30-second sleep interval, whereas generic API errors pause execution for 10 seconds. Authentication errors terminate execution immediately:

```
except RateLimitError:
    if attempt == retries - 1:
        raise
    time.sleep(30)
except AuthenticationError:
    raise RuntimeError(
        "Errore di autenticazione: controlla OPENAI_API_KEY."
```

```
    )
except APIError:
    if attempt == retries - 1:
        raise
    time.sleep(10)
```

For each annotated response, caption and foil generation proceed sequentially. The former accepts the question and human answer as input, whereas the latter accepts the generated caption along with its associated macroarea:

```
caption = generate_caption(
    row["question_text"],
    row["risposta"],
)
foil = generate_foil(
    caption,
    row["macroarea"],
)
```

Outputs are post-processed via `clean_generation()`, which strips optional `S:` or `F:` prefixes returned by the model:

```
def clean_generation(text, prefix):
    text = text.strip()
    if text.startswith(f"{prefix}: "):
        text = text[3:]
    elif text.startswith(f"{prefix}:"):
        text = text[2:]

    return text.strip()
```

To accommodate long-running executions, the script implements a checkpointing mechanism. Cached entries are reused only if their question, annotator, and human answer match the current source data and both generated fields are non-null:

```
checkpoint_is_valid = (
    saved is not None
    and str(saved["question"]) == str(row["question_text"])
    and str(saved["annotator"]) == str(row["annotator"])
    and str(saved["human_answer"]) == str(row["risposta"])
    and pd.notna(saved["caption"])
    and pd.notna(saved["foil"])
)
```

Newly generated entries are appended incrementally to `caption_foil_checkpoint.csv`:

```
pd.DataFrame(generated_rows).to_csv(
    CHECKPOINT_FILE,
    index=False,
    encoding="utf-8-sig",
)
```

Following generation, caption and foil positions are randomized independently within each question category. A fixed random seed guarantees reproducibility while maintaining a balanced distribution of options across positions:

```
rng = random.Random(RANDOM_SEED)
for category, indices in (
    df.groupby("question_category").groups.items()
):
    indices = list(indices)
    n = len(indices)
    labels = [0] * (n // 2) + [1] * (n // 2)

    if n % 2:
        labels.append(rng.randint(0, 1))

    rng.shuffle(labels)
```

The resulting `target` variable indicates the position of the correct caption: a value of 0 places the caption in `answer1`, whereas a value of 1 assigns it to `answer2`:

```
df["answer1"] = df.apply(
    lambda row:
    row["caption"]
    if row["target"] == 0
    else row["foil"],
    axis=1,
)
df["answer2"] = df.apply(
    lambda row:
    row["foil"]
    if row["target"] == 0
    else row["caption"],
    axis=1,
)
```

The final dataset is saved to `data/vsv/caption-foil/caption_foil.csv`. Alongside the randomized options and target labels, the output retains all metadata required to trace each pair back to its originating video, question, annotator, and human response:

```
columns = [
    "id",
    "video_id",
    "video_name",
    "pool_id",
    "pool_item",
    "question_category",
    "question",
    "annotator",
    "human_answer",
    "caption",
```

```
    "foil",
    "answer1",
    "answer2",
    "target",
]
final_df[columns].to_csv(
    OUTPUT_FILE,
    index=False,
    encoding="utf-8-sig",
)
```

Caption–Foil Quality Validation

The automatic generation of a foil does not, by itself, guarantee that the resulting sentence constitutes a valid negative counterpart of the corresponding caption. Although generation is constrained through category-specific prompts, the model may introduce changes that are semantically unrelated to the targeted reasoning phenomenon, alter more information than necessary, or produce a sentence that remains compatible with the original caption. For this reason, the generated *caption–foil* pairs undergo an additional semi-automatic validation stage before being included in the final VSV evaluation set. The validation procedure follows the two-step strategy adopted in *MAIA*, adapted here to the three reasoning dimensions considered in *MAIA-AV*. The first stage is a *structural check*, whose purpose is not simply to determine whether the caption and foil differ, but to verify that their difference concerns the semantic dimension the corresponding question is meant to evaluate. This distinction is fundamental for preserving the diagnostic nature of the benchmark. If, for instance, a temporal foil modified the identity of an entity rather than the ordering of two events, a model could distinguish the two statements through entity recognition alone, without any temporal reasoning. Similarly, a causal pair would stop providing a controlled test of causal understanding if the foil simultaneously changed the cause, the participants, and the location of the event. Structural verification is what keeps the contrast introduced during foil generation aligned with the competency each question

is meant to probe. For **causal** instances, validation requires the foil to modify the cause, the effect, or the causal relation expressed by the caption, while preserving the remaining content as much as possible. For **temporal** instances, the contrast must concern temporal information, including event ordering, before–after relations, or the moment at which an event occurs. The **spatial** category calls for a slightly broader criterion. Unlike the original *MAIA* taxonomy, where spatial reasoning is split into Total and Partial subcategories, the spatial questions in *MAIA-AV* combine phenomena from both settings. The validation prompt therefore accepts changes involving locations, positions, directions, and relative spatial relations, including relations that hold only during a particular phase of the video. This broader formulation avoids forcing the new dataset into a distinction its annotation scheme does not retain, while keeping the core requirement that the foil differ from the caption specifically in its spatial content. Pairs that satisfy this first criterion move on to a second validation step based on *Natural Language Inference* (NLI), where the caption is treated as the premise and the foil as the hypothesis. The model assigns one of three standard NLI labels: *Entailment*, *Contradiction*, or *Neutral*. The expected relation is *Contradiction*, since the foil is designed to offer an alternative statement that should not hold under the same interpretation as the caption. This step addresses a different source of error than the structural check. A foil may correctly manipulate the intended semantic dimension while still failing to create a sufficiently distinct alternative. Conversely, a sentence may differ substantially from the caption yet fail the structural test because the modification does not isolate the phenomenon under evaluation. The NLI result is nevertheless treated as a diagnostic signal rather than an infallible automatic decision, particularly for causal and, to a lesser extent, spatial statements. Two alternative causes, for example, may be logically compatible in isolation even though only one is intended to describe the event represented in the dataset. In such cases a general-purpose NLI model may classify the relation as *Neutral* rather than *Contradiction*. A similar pattern was observed during the original *MAIA* validation, where qualitative inspection showed that many pairs labelled as neutral were nevertheless valid caption–foil contrasts. For this reason, only pairs that pass the structural check and receive the *Contradiction* label are accepted automatically. Structurally valid pairs classified as *Neutral* are marked for manual review, while *Entailment*

cases are given higher review priority, since they provide stronger evidence that the foil may remain semantically compatible with the caption. Category-sensitive structural verification and category-independent semantic inference thus provide two complementary safeguards. The former preserves the correspondence between each pair and the reasoning ability it is meant to test. The latter checks whether the resulting negative statement establishes an adequate semantic opposition with the corresponding positive statement. Importantly, neither stage is used to silently discard uncertain data. Every instance flagged as structurally invalid, entailed, neutral, or affected by execution errors is isolated in a dedicated review set, manually inspected, and, whenever the validation reveals an actual problem, corrected so that the intended semantic contrast is restored. The corrected pair is then run back through the same validation procedure. Human verification at this final stage keeps automatic classifier errors from propagating into the benchmark, while the pipeline as a whole remains predominantly automated.

Automatic Validation and Manual Revision

The automatic validation procedure was used as a quality-control step to identify caption–foil pairs that required further inspection. The structural check evaluated whether the foil modified the semantic dimension associated with the corresponding category, while the NLI stage assessed the relation between caption and foil in terms of *Entailment*, *Contradiction*, or *Neutral*. The output of these automatic checks was not treated as a definitive decision on the validity of each pair. In particular, some instances labelled as *FAIL_STRUCTURAL*, *Neutral*, or *Entailment* could still represent acceptable contrasts when considered in relation to the original question, the human answer, and the intended semantic category. This is especially relevant for cases in which the contrast is expressed through the answer-bearing element itself, such as a change of location in spatial questions or a change in the participant occupying a specific temporal position. Similarly, in causal questions, two alternative explanations may be classified as *Neutral* by an NLI model even when only one of them is supported by the video. For this reason, the complete caption–foil file was subsequently revised manually. Each automatically flagged instance was inspected together with its question, human answer, semantic category, caption, and

foil. The automatic labels were used only to prioritize potentially problematic cases, not to determine whether a pair had to be discarded or modified. When a caption–foil pair was considered semantically valid and sufficiently consistent with the intended contrast, it was retained without changes. Whenever the manual inspection revealed an actual issue, such as a foil targeting the wrong semantic dimension, an ambiguous or insufficiently discriminative contrast, a formulation that did not adequately reflect the original human answer, or a mismatch between the question and the generated statements, the pair was manually corrected instead. The final dataset thus reflects a combination of automatic quality-control signals and human verification. Modifications were introduced only when they were considered necessary after direct inspection of the corresponding instance.

Implementation Details The validation procedure is implemented as a separate post-processing stage operating on the output of the caption–foil generation pipeline. Its input is the complete dataset stored in `data/vsv/caption-foil/caption_foil.csv`. Validation is performed on the canonical `caption` and `foil` fields rather than on the randomized `answer1` and `answer2` columns used by the final VSV task, since option randomization affects only presentation order and must not interfere with the semantic relation being assessed. The relevant paths and expected semantic categories are defined explicitly:

```
INPUT_PATH = Path(
    "data/vsv/caption-foil/caption_foil.csv"
)
OUTPUT_DIR = Path(
    "data/vsv/caption-foil/validation"
)

STRUCTURAL_OUTPUT = (
    OUTPUT_DIR /
    "caption_foil_structural_validation.csv"
)

FINAL_OUTPUT = (
```

```
    OUTPUT_DIR /
    "caption_foil_validation.csv"
)

VALID_CATEGORIES = {
    "causale",
    "temporale",
    "spaziale",
}
```

Before any API request is issued, the script verifies the availability of the fields required for validation and checks that all category labels belong to the three supported macroareas:

```
REQUIRED_COLUMNS = {
    "id",
    "question_category",
    "caption",
    "foil",
}

missing_columns = (
    REQUIRED_COLUMNS - set(df.columns)
)

if missing_columns:
    raise ValueError(
        "Mancano le seguenti colonne nel CSV: "
        + ", ".join(sorted(missing_columns))
    )
```

The first validation stage selects a category-specific prompt according to the value stored in `question_category`. All three prompts share the same decision format but define validity according to different semantic constraints. Causal pairs, for instance, explicitly

require the modification to concern causal information:

```
if category == "causale":
    return (
        "Given an Italian caption (C) dealing "
        "with causal relations between events "
        "and its foil (F), your task is to assess "
        "whether F is a valid causal foil of C.\n\n"

        "To be valid, F must modify the cause, "
        "the effect, or the causal relation "
        "expressed in C while preserving the "
        "remaining relevant content as much "
        "as possible.\n"

        "If F is a valid causal foil, output "
        "'correct'. Otherwise output "
        "'not correct'.\n\n"

        f"C: {caption}\n"
        f"F: {foil}\n"
        "Output:"
    )
```

The temporal prompt follows the same structure but constrains the acceptable difference to event ordering or other temporal properties:

```
if category == "temporale":
    return (
        "To be valid, F must modify the temporal "
        "information expressed in C, such as "
        "the ordering of events, before/after "
        "relations, the moment at which an event "
        "occurs, or another temporally relevant "
```

```
        "relation.\n"
    )
```

For spatial instances, the prompt is deliberately formulated to cover both stable locations and time-dependent spatial configurations:

```
if category == "spaziale":
    return (
        "The spatial category considered here "
        "includes both locations and spatial "
        "relations, including cases in which "
        "spatial information refers to a specific "
        "moment or phase of the video.\n\n"

        "To be valid, F must modify spatial "
        "information expressed in C, such as "
        "the location, position, direction, or "
        "spatial relation of a person, object, "
        "or event.\n"
    )
```

The model is constrained to produce only one of two labels, `correct` or `not correct`. Outputs are normalized before being stored, to remove minor formatting variations:

```
def normalize_structural_label(text):
    value = clean_text(text).lower()

    if value.startswith("output:"):
        value = value[len("output:"):].strip()

    value = value.rstrip(".").strip()

    if value == "not correct":
        return "not correct"
```

```
    if value == "correct":
        return "correct"

    return "invalid"
```

The structural result is saved for every caption–foil pair in the `structural_eval` field. Incremental serialization is performed after each processed instance, so that completed evaluations survive an interruption:

```
df.at[index, "structural_eval"] = result

df.to_csv(
    STRUCTURAL_OUTPUT,
    index=False,
)
```

The second stage applies only to pairs that pass structural validation. Caption and foil are mapped respectively to the NLI premise `S1` and hypothesis `S2`:

```
prompt = (
    "Your task is to determine the natural "
    "language inference (NLI) relationship "
    "between S1 and S2.\n\n"

    "The possible labels are:\n"
    "- Entailment: S2 logically follows "
    "from S1.\n"
    "- Contradiction: S2 contradicts S1.\n"
    "- Neutral: S1 and S2 are related, "
    "but S2 neither follows from nor "
    "contradicts S1.\n\n"

    f"S1: {caption}\n"
```

```
    f"S2: {foil}\n"
    "Output:"
)
```

The resulting output is normalized to one of the three expected NLI labels:

```
if value == "contradiction":
    return "Contradiction"

if value == "neutral":
    return "Neutral"

if value == "entailment":
    return "Entailment"
```

The two validation signals are then combined into an explicit status variable, with automatic acceptance limited to pairs that are structurally valid and classified as contradictions:

```
def get_validation_status(
    structural_eval,
    nli_eval,
):
    if structural_eval != "correct":
        return "FAIL_STRUCTURAL"

    if nli_eval == "Contradiction":
        return "PASS"

    if nli_eval == "Neutral":
        return "REVIEW"

    if nli_eval == "Entailment":
        return "REVIEW_HIGH_PRIORITY"

```

```
    return "ERROR"
```

Pairs that fail the structural check do not proceed automatically to semantic acceptance and are marked as `FAIL_STRUCTURAL`. Structurally valid pairs classified as `Neutral` are assigned `REVIEW`, while `Entailment` produces `REVIEW_HIGH_PRIORITY`. This hierarchy keeps uncertain model judgements from being treated as definitive dataset errors. At the end of the procedure, all non-passing cases are extracted into a dedicated manual-review file:

```
review_mask = df[
    "validation_status"
].isin(
    [
        "REVIEW",
        "REVIEW_HIGH_PRIORITY",
        "FAIL_STRUCTURAL",
        "ERROR",
    ]
)

review_df = df.loc[
    review_mask
].copy()

review_df.to_csv(
    OUTPUT_DIR /
    "caption_foil_manual_review.csv",
    index=False,
)
```

This file serves as the interface between automatic and human validation. Each flagged instance is manually inspected by comparing its caption, foil, question, and semantic category. Cases where the automatic validator is overly conservative but the contrast is in

fact appropriate are retained unchanged. When inspection instead confirms that the foil modifies the wrong semantic property, remains compatible with the caption, introduces unrelated information, or otherwise fails to provide the intended minimal contrast, the foil is corrected by hand. Corrected instances are then re-evaluated through the same structural and NLI checks before being incorporated into the final dataset. The complete validation output retains the original metadata alongside `structural_eval`, `nli_eval`, and `validation_status`. This preserves full traceability from each final VSV item back to its generated caption, foil, question, annotator response, automatic validation decisions, and any manual correction.

3. Study Design and Experimental Setup

3.1 Overview of the Evaluated Models

Qwen Family

The Qwen models considered in this work share an omnimodal architecture designed to process heterogeneous information within a unified computational framework. Both Qwen2.5-Omni and Qwen3-Omni accept textual, visual, and acoustic inputs, with modality-specific encoders transforming raw signals into representations compatible with a shared language-model backbone. Multimodal integration takes place after perceptual encoding, and particular attention is paid throughout the family to the temporal coordination of audio and video [24, 25].

Qwen2.5-Omni-3B

Qwen2.5-Omni introduces the *Thinker-Talker* architecture, separating multimodal understanding from speech generation. The *Thinker* processes the input modalities and produces textual representations and responses, while the *Talker* takes the Thinker's internal representations and generates speech from them. Perceptual understanding and spoken generation thus remain functionally distinct components of an otherwise end-to-end pipeline [24]. Text is processed through the Qwen tokenizer. Audio is resampled to 16 kHz and converted into a 128-channel mel-spectrogram, with the encoder producing representations at a temporal resolution of roughly 40 ms. Visual information is handled by a Vision Transformer derived from Qwen2.5-VL and shared across image and video inputs; frames are sampled dynamically to preserve relevant visual content while matching the temporal granularity of the acoustic stream [24]. The modality-specific encoders are limited to perceptual extraction: their output is passed to the *Thinker*, where multimodal information is processed within the shared Transformer. Both audio and visual encoders additionally support block-wise processing, allowing long multimodal sequences to be handled incre-

mentally [24]. Multimodal alignment relies on *Time-aligned Multimodal Rotary Position Embedding* (TMRoPE), which represents positional information along temporal, vertical, and horizontal dimensions. Text is assigned identical positional values across all three; images retain a constant temporal coordinate while preserving spatial position along height and width; audio receives temporal positions tracking its progression through the acoustic stream [24]. Video combines both axes: each frame keeps its two-dimensional spatial layout while carrying a temporal coordinate tied to its position in the clip, on a scale coordinated with the acoustic representation. Audio and video can thus be placed within a common temporal reference system, which is what allows the *Thinker* to relate acoustic and visual evidence occurring at corresponding moments. Encoded representations from both streams are arranged within a single multimodal sequence and processed jointly by the language model's attention mechanism, so integration occurs before response generation rather than through a late combination of independent modality-specific predictions [24]. The *Talker* is a dual-track autoregressive Transformer decoder, conditioned on the high-dimensional representations it receives from the *Thinker*; the resulting discrete speech units are converted into a waveform through a sliding-window diffusion Transformer [24]. The technical report describes this organization at the level of the Qwen2.5-Omni family as a whole, without a fusion mechanism specific to the 3B variant [24].

Qwen3-Omni-30B-A3B-Instruct

Qwen3-Omni keeps the Thinker-Talker organization of its predecessor while redesigning several components. It handles text, images, audio, and video, and supports both textual and spoken generation; in the 30B-A3B configuration, both the *Thinker* and the *Talker* adopt a Mixture-of-Experts architecture, increasing overall capacity while limiting the parameters activated at each inference step [25]. The *Thinker* still carries out multimodal understanding and text generation by combining representations from dedicated perceptual encoders. The *Talker*, however, is more loosely coupled to it than in Qwen2.5-Omni: rather than depending directly on the Thinker's high-level textual hidden states, it conditions speech generation on textual context together with high-dimensional visual and acoustic representations [25]. Audio is processed by the *Audio Transformer* (AuT). Raw

audio is resampled to 16 kHz and represented as a 128-channel mel-spectrogram; convolutional downsampling brings the resulting representations to roughly 12.5 Hz, or about 80 ms per acoustic frame. AuT uses attention windows of different sizes to support both global audio understanding and block-wise processing [25]. Visual input is processed by an encoder derived from Qwen3-VL and initialized from SigLIP2-So400m, shared across images and video; as in Qwen2.5-Omni, frames are sampled dynamically to preserve visual content while matching the temporal granularity of the audio stream [25]. TMRoPE is retained for multimodal alignment, again along temporal, vertical, and horizontal dimensions: text keeps identical coordinates across all three, images hold a constant temporal coordinate while preserving spatial position, and audio now receives absolute temporal identifiers at roughly 80 ms resolution [25]. Video frames are assigned temporal identifiers taken from their actual timestamps, on the same scale used for audio, so acoustic and visual representations share an absolute temporal coordinate system while spatial information is still carried separately through height and width [25]. This replaces the fixed temporal chunking of Qwen2.5-Omni with alignment based on absolute temporal identifiers, producing a finer correspondence between acoustic and visual events and removing the dependence on a predefined chunk size [25]. The aligned representations are then processed by the Mixture-of-Experts *Thinker*, so multimodal integration occurs within the shared Transformer computation after perceptual encoding and temporal alignment; the pre-training strategy reinforces this from the outset, drawing on unimodal and cross-modal data spanning image-text, audio-text, video-text, video-audio, and video-audio-text combinations [25]. Speech generation is likewise redesigned. The *Talker* predicts a multi-codebook acoustic representation based on residual vector quantization: the first codebook is generated autoregressively, while a Multi-Token Prediction module estimates the remaining residual codebooks. The final waveform is produced by *Code2Wav*, a causal convolutional renderer built for incremental synthesis [25].

Gemma Family

The Gemma models considered in this work implement multimodality through two distinct strategies. Gemma 3n E4B relies on dedicated perceptual encoders that convert

acoustic and visual signals into representations processed by the language-model backbone. Gemma 4 12B instead adopts a unified, encoder-free architecture, mapping raw perceptual information directly into the decoder's embedding space through lightweight projection modules [18, 6].

Gemma 3n E4B IT

Gemma 3n accepts text, images, audio, and video and generates textual responses; separate perceptual encoders convert acoustic and visual signals into token representations for the central language model [18]. The E4B configuration is built on *MatFormer* (Matryoshka Transformer), which nests a smaller E2B computational path within the larger E4B structure; both are trained jointly, allowing the model to operate under different computational budgets without requiring separate weights [18]. Gemma 3n also introduces *Per-Layer Embeddings*, storing a substantial portion of the embedding parameters outside accelerator memory. As a result, E4B keeps only around four billion core parameters resident on the accelerator despite a larger overall parameter count ,a gain in computational efficiency rather than a change to how modalities are integrated [18]. Audio is processed by an encoder based on the Universal Speech Model, producing roughly one acoustic token every 160 ms (about six tokens per second) that feed directly into the language model [18]. Visual input is handled by MobileNet-V5-300M, supporting resolutions of 256×256, 512×512, and 768×768 pixels across both image and video content [18]. A *Multi-Scale Fusion VLM adapter* processes the resulting visual representations before they reach the language model; its role is limited to combining features at different scales and does not amount to a general audiovisual fusion mechanism [18]. Multimodal integration remains centered on the language-model backbone: audio and visual encoders produce modality-specific representations that become available to the Transformer during generation, and the available documentation does not describe a shared temporal alignment mechanism between acoustic and visual signals comparable to TMRoPE in the Qwen models [18]. Gemma 3n further employs *KV Cache Sharing*, letting keys and values from intermediate attention layers be reused by subsequent ones, which reduces redundant computation during the prefill stage, a benefit that is particularly relevant for the long sequences produced

by audio and video input [18].

Gemma 4 12B IT

Gemma 4 is a natively multimodal family built on a decoder-only Transformer, and the 12B configuration departs from the rest of the family architecturally. Rather than relying on perceptual encoders, it is trained from scratch under a unified encoder-free paradigm, with visual and acoustic input mapped directly into the language model's embedding space through lightweight projection modules [6]. This sets it apart from the other Gemma 4 configurations: the smaller E2B and E4B models retain distinct vision and audio encoders, and even the larger variants keep dedicated visual processing. Gemma 4 12B alone drops both the vision and the audio encoder, replacing them with a direct projection of raw perceptual information [6]. For visual input, the model operates on RGB patches of $48 \times 48 \times 3$ values. A large linear projection of roughly 35 million parameters takes the place of the Vision Transformer used elsewhere in the family, mapping the resulting representations directly into the language model's embedding space [6]. Spatial information is preserved by adding two-dimensional coordinate-based positional embeddings to the projected patches, followed by a final LayerNorm before the visual representations enter the backbone, enough to keep spatial structure available without a separate visual encoder [6]. The audio pathway follows the same principle. Raw audio, sampled at 16 kHz, is divided into 40 ms segments of 640 values each; the USM-based Conformer encoder used by the other Gemma 4 variants is removed entirely, and each 640-dimensional segment is projected directly into the language model's embedding space [6]. No additional positional encoding is introduced for these acoustic representations, since their sequential order already preserves the temporal structure of the audio stream. Visual and acoustic information thus enter the same embedding space through modality-specific linear projections rather than through independent high-capacity encoders [6]. Multimodal integration accordingly takes place directly inside the decoder-only Transformer: the projection modules establish representational compatibility between raw perceptual input and the language model's embedding space, and the Transformer itself carries out the subsequent interaction between perceptual and linguistic information, narrowing the separation

between perceptual encoding and language processing found in more conventional multimodal systems [6]. The 12B model is dense and combines local and global self-attention, with five local attention blocks per global block. Local layers use RoPE, while global layers use p-RoPE with $p = 0.25$; the architecture also reuses keys as values in global attention layers, reducing the memory footprint of the global KV

3.2 Preliminary Analysis as a Diagnostic Tool for Multimodal Assessment

The preliminary analysis aims to determine which information relevant to the subsequent resolution of *MAIA-AV* questions can be autonomously recovered by each model from the visual content of the video alone. Consequently, the analysis is conducted in a *question-independent* setting: the model receives solely the video input, with no access to the questions, corresponding answers, audio streams, or transcriptions. The output is a structured and temporally grounded representation of the visual content, which is only later compared with the informational requirements of individual questions. This separation distinguishes the model's capacity to extract visual information from its ability to retrieve relevant evidence when guided by a linguistic query. While the preliminary analysis does not directly measure performance on the VSV task, its purpose is to characterize what the model can represent prior to the introduction of a specific question-answering objective. This distinction addresses a key limitation of evaluations based solely on final accuracy. A correct prediction does not guarantee that the model has correctly identified relevant entities, reconstructed events, or established the relations necessary to support its output. Conversely, an incorrect answer fails to reveal which stage of the process caused the failure. The preliminary analysis thus introduces an intermediate diagnostic layer between the raw video content and the final task outcome. Methodologically, this approach relates to *VILMA* [10], specifically regarding its distinction between proficiency tests and main tests. In *VILMA*, proficiency tests evaluate fundamental capabilities deemed necessary for resolving complex temporal phenomena via a zero-shot caption-foil paradigm. While our method does not replicate this specific evaluation protocol, it adopts the core underlying principle: com-

plex performance must be interpreted relative to the prerequisite capabilities that underpin it. In *MAIA-AV*, these prerequisites are examined via a structured representation of the visual content. For each video, four multimodal and omnimodal models, subsequently evaluated on the VSV task, independently analyze temporally localized video segments. Outputs are initially kept separate to enable cross-representation comparison without prematurely merging predictions. Inter-model agreement is interpreted as an indicator of **prediction stability** rather than absolute ground truth, as multiple models may converge on an identical yet unsupported interpretation. The representation is constructed progressively across three levels: **entities**, **events**, and **relations between events**. This hierarchy decouples directly observable perceptual information from information requiring temporal integration or higher-level inference. The **entity level** describes elements identified within the visual content and their observable properties. It details the objects, individuals, and other scene components detected by the model, along with their corresponding temporal intervals of presence. The **event level** captures actions, interactions, and state changes tied to specific temporal intervals. Because overlapping video segments may yield duplicate descriptions of the same event, the extracted outputs are consolidated to eliminate redundancy while preserving genuinely recurrent events. Temporal anchoring maintains a direct link between each event and the corresponding video segment providing supporting evidence. The **relation level** organizes extracted events according to temporal and inferential dependencies. Temporal relations encompass configurations such as *before*, *after*, *during*, and *overlap*, supporting event reordering and tracking changes across the video timeline. Causal relations are treated independently of temporal succession, simple temporal precedence does not inherently establish a cause-effect relationship. The analysis thus differentiates causal information directly supported by visual evidence from relations requiring further inference over events, states, or intentions. This distinction separates observable content from information introduced by the model's inferential process. The distinction between representational levels is crucial because questions assigned to the same nominal category can demand substantially different types and volumes of evidence. For instance, identifying an object's location within a single frame requires fundamentally different operations than determining its position following a sequence of movements. Similarly, rec-

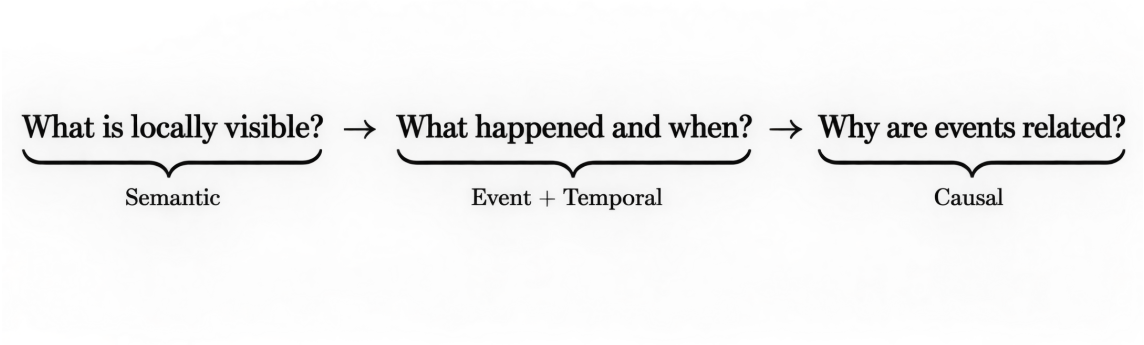

Figure 3.1: Conceptual schema of the preliminary analysis.

ognizing an isolated event differs from reconstructing the sequence of multiple events or inferring causal links between them. To systematically categorize these distinctions, questions are analyzed along three operational dimensions: **spatial**, **temporal**, and **causal**. A unified three-level scale is applied across each dimension: Level 0 denotes cases where the required information can be directly retrieved, Level 1 entails integrating an explicit relation or a limited number of evidence units and Level 2 demands broader reconstruction across multiple events, temporal phases, state changes, or implicit relations. These criteria are summarized in Table 3.1. This classification reflects the informational requirements of the questions rather than their linguistic complexity. It accounts for properties such as the temporal distribution of evidence, the number of relevant evidence units, the presence of inter-event relations, and whether the required information is explicit or implicit. Once constructed independently, the visual representation can be compared against the question requirements. This comparison determines whether the entities, events, and relations necessary to answer a question are already captured within the vision-only representation. The evaluation thus does not depend directly on the model's capacity to generate the correct answer, instead, it assesses whether the supporting information is present within its visual representation. This step is particularly relevant for *MAIA-AV*, as it establishes a visual baseline prior to introducing additional modalities. Prior findings [4] demonstrated that, in the original *MAIA* setting, incorporating acoustic information yielded no systematic performance gains, while including transcriptions led to a slight yet statistically robust decline in accuracy. This aligns with the dataset's original design, where questions were

formulated primarily around visual content rather than information explicitly conveyed via audio. The preliminary analysis thus establishes *what can be seen* prior to evaluating the contribution of *what can be heard*. This clarifies whether the visual channel inherently contains the required evidence and highlights categories where visual information alone proves insufficient. Consequently, the incremental contributions of audio and transcriptions can be assessed against an explicit visual baseline rather than solely through shifts in downstream task accuracy. Overall, the preliminary analysis transforms *MAIA-AV* from a purely result-driven evaluation into a diagnostic, competence-oriented framework. Final performance can thus be analyzed across specific sub-capabilities, including entity recognition, event identification, temporal integration, spatial reconstruction, and causal inference, thereby disentangling evidence availability failures from errors in downstream selection or integration during the VSV task.

Category	**Level 0**	**Level 1**	**Level 2**
Spatial (S)	Direct retrieval of an entity's location or spatial property from a single frame.	Explicit spatial relation or event-conditioned localization involving at most two evidence units.	Spatial reconstruction across multiple phases, including movement, relocation, or state change.
Temporal (T)	Recognition of a single event or state without temporal comparison.	Temporal localization or pairwise ordering between two explicit events.	Multi-event temporal reconstruction, including duration, repetition, onset, or state evolution.
Causal (C)	Retrieval of an explicitly stated cause without inference.	Explicit cause-effect relation involving at most two evidence units.	Implicit or multi-event causal reconstruction requiring contextual or intentional inference.

Table 3.1: Complexity levels for spatial, temporal, and causal questions.

Video Preprocessing: Temporal Segmentation and Frame Sampling

Prior to the entity extraction phase, each video is preprocessed by dividing it into four-second temporal windows with a stride of three seconds. Consecutive spans share one second of visual content. This overlap reduces the risk that an action or state change occurring near the boundary between two segments is only partially observed. Simultaneously, it maintains shared context across consecutive observations without altering the chronological order of the video. For each segment, the `start_time` and `end_time` are preserved, keeping each segment associated with its original temporal position in the video. This information is maintained throughout all subsequent stages of the pipeline, enabling observations to be reconnected with their exact position in the video. The windowing process serves exclusively to provide temporal indexing and structure visual evidence, leaving content and semantic interpretation to subsequent phases. During this preprocessing stage, redundant information is deliberately retained to prioritize continuity of evidence, with redundancy resolution deferred to the output consolidation step. Ultimately, the preprocessing phase yields an ordered sequence of temporally localized visual units that serve as the shared foundation for all subsequent stages of analysis.

Implementation Details The preprocessing stage is implemented in `video_preprocessing.py` to convert each source video into a set of temporally indexed visual representations. The script first retrieves core metadata via `ffprobe`, including duration, frame rate, spatial resolution, codec, and audio stream presence. Although audio metadata is recorded, the acoustic signal is excluded from visual representation construction. Frames are extracted using OpenCV by seeking directly to target temporal positions. Each frame is saved as a JPEG image with its original timestamp embedded in both the filename and the JSON manifest, preserving temporal provenance independently of downstream processing. A core output of the script is the dense visual stream, sampled by default at 4 FPS:

```
parser.add_argument("-dense-fps", type=float, default=4.0)
```

Extracted frames are saved in the `dense_frames` directory for each video. Simultaneously,

the script generates 32 uniform global frames and performs shot-boundary detection via PySceneDetect. These boundaries define variable-length segments with a target duration of 4 seconds (ranging from 2 to 6 seconds) and a default overlap of 0.5 seconds, extracting eight frames per segment. The preprocessing output is serialized into `metadata.json`, `frames.json`, `shots.json`, and `segments.json`. For the subsequent preliminary analysis, semantic extraction scripts operate directly on the dense stream rather than shot-based segments. Utility functions locate each video's `dense_frames` directory, parse timestamps from filenames, and sort frames chronologically. The model thus receives temporally ordered visual observations instead of isolated or semantically segmented images. The inference temporal windows are configured as follows:

```
parser.add_argument("-window-seconds", type=float, default=4.0)
parser.add_argument("-stride-seconds", type=float, default=3.0)
```

A 4-second window with a 3-second stride yields a 1-second overlap between consecutive observations. The pipeline selects dense frames within these nominal boundaries. If frame counts exceed the allowed maximum, frames are sampled uniformly across the window. At 4 FPS, a 4-second interval contains 16 frames, remaining well below the default 32-frame ceiling. Duplicate consecutive windows are automatically omitted. Each temporal unit is encapsulated in a `TemporalWindow`, mapping a unique segment ID to its start time, end time, and frame paths. Crucially, recorded boundaries reflect the timestamps of the first and last selected frames rather than nominal window limits. Millisecond precision is maintained to ensure exact alignment with the source video. The transition to semantic extraction relies strictly on *ordered dense frames*. For each window, the model receives selected RGB images alongside a prompt specifying the video ID, temporal interval, and frame-to-timestamp map. No questions, answers, transcriptions, or acoustic signals are provided. Temporal segmentation thus serves purely as an indexing mechanism for visual evidence, deferring semantic interpretation to the subsequent extraction stage.

Semantic Representation Extraction

Building upon the defined temporal windows, the pipeline proceeds with semantic representation extraction. This stage generates a structured semantic description for each window, processing every window independently. The extraction procedure is repeated across all four evaluated models. Although the models employ distinct architectures and inference modalities, each receives identical inputs and is constrained to produce a standardized representation adhering to a common schema, as shown in Table3.2., adhering to a common schema. Consequently, performance differences can be attributed directly to model capabilities rather than annotation formatting. For each window, the representation spans seven semantic categories:

entities : Operating at the most fundamental descriptive level, each entity receives a local identifier, a concise label, visually observable attributes, and its role within the scene when discernible. The representation tracks the temporal occurrence of the entity and the specific frames providing visual evidence. These local identifiers enable consistent references across other descriptive categories within the same segment.

actions : Actions represent visually observable occurrences, specifying the performing agent and target object whenever feasible.

events : Events denote broader semantic units linked to one or more participants across a given temporal span. This distinction separates the detection of an isolated action from the broader event context in which it occurs.

spatial_relations : Spatial configurations between entities or scene elements are defined using a subject-relation-object structure. Permissible relations include configurations such as *in front of*, *behind*, *above*, *below*, *inside*, *outside*, *left of*, *right of*, *near*, and *on*. These spatial descriptors are temporally localized, meaning a spatial configuration observed in one segment is not assumed to hold throughout the video.

state_changes : State changes capture observable transformations affecting an entity. By contrasting a prior state with a subsequent state, this structure represents tempo-

ral variations within a segment. This level is essential when answering a question requires tracking changes in an entity's properties, position, or condition over time.

temporal_relations : The model describes temporal links such as *before*, *after*, *overlaps*, *during*, *starts*, *finishes*, and *simultaneous*. However, these relations remain strictly localized to the current window. Global video temporal structure is not reconstructed at this stage, nor are event correspondences across different segments systematically resolved.

causal_hypotheses : Causal candidate relations are explicitly separated from directly observed facts. The framework requires causal hypotheses to be flagged as inferred evidence, preventing temporal precedence from being automatically interpreted as a cause and effect relationship. This design preserves a clear boundary between observed evidence and model inference.

The distinction between observed and inferred evidence is preserved through the *evidence_type* field, accompanied by a confidence score between 0 and 1. Each element also includes an *evidence_frames* list identifying supporting frames. During output normalization, frame references are retained only if they fall within the processed window, and generated temporal intervals are similarly bounded by segment limits. These constraints maintain strict alignment between semantic representations and supporting visual evidence. Categories may remain empty if a model detects insufficient evidence within a segment. At the segment level, the pipeline preserves *segment_id*, *start_time*, *end_time*, and *input_frames*. At this initial stage, a unified global video description is not yet constructed. Due to frame overlap between consecutive windows, identical entities, actions, or events may be extracted repeatedly across adjacent segments. This redundancy is intentionally preserved during extraction to maintain a direct mapping between each prediction and its supporting local evidence. Entity reconciliation and global temporal aggregation are deferred to subsequent pipeline stages.

Implementaion Detail Entity and semantic analysis is implemented through a unified extraction pipeline shared across all evaluated models. Model-specific wrapper scripts

Field	Information represented
`entities`	Identified entities, observable attributes, roles, and temporal occurrence.
`actions`	Local actions, their actors, and involved objects.
`events`	Temporally extended local events and their participants.
`spatial_relations`	Spatial configurations between entities or scene elements.
`state_changes`	Observable transitions between an entity's previous and subsequent states.
`temporal_relations`	Temporal relations between events identified within the same local context.
`causal_hypotheses`	Candidate causal relations explicitly marked as inferred evidence.

Table 3.2: Semantic information extracted from each temporal window.

load the corresponding architectures and format selected input frames, while temporal framing, prompt generation, schema definitions, validation logic, and serialization are centralized in `utils.py`. This separation ensures that all models are evaluated against an identical semantic protocol regardless of underlying architectural differences. The common semantic space is defined across seven categories:

```
SEMANTIC_CATEGORIES = (
    "entities",
    "actions",
    "events",
    "spatial_relations",
    "state_changes",
    "temporal_relations",
    "causal_hypotheses",
)
```

The categories illustrated in Table 3.2 specify the structured information required from

each temporal window. For each temporal window, the pipeline constructs a standardized extraction prompt via `build_semantic_prompt`. In addition to category definitions, the prompt provides the video ID, window boundaries, and an explicit frame-to-timestamp map:

```
frame_map = "\n".join(
    f'- frame {index}: "{path.name}", '
    f'timestamp={get_timestamp(path):.3f}s'
    for index, path in enumerate(
        window.frame_paths,
        start=1,
    )
)
```

Rather than eliciting free-form descriptions, the prompt[1] enforces strict visual grounding. Every extracted element must reference supporting frames, temporal boundaries, a confidence score, and an `evidence_type`. Directly visible details are classified as `observed`, whereas inferential claims are labeled as `inferred`. Furthermore, the prompt explicitly instructs the model not to treat simple temporal succession as causality:

```
    """Non inventare dettagli non visibili.
    Usa evidence_type="observed"
    per fatti direttamente visibili.
    Usa evidence_type="inferred"
    soltanto per inferenze inevitabili
    o ipotesi causali.
    Per ogni elemento indica confidence
    tra 0 e 1 e i nomi esatti dei frame
    che lo supportano.
```

[1]English Translation:*"Do not invent details that are not visible. Use evidence_type="observed" for directly visible facts. Use evidence_type="inferred" only for unavoidable inferences or causal hypotheses. For each element, indicate confidence between 0 and 1 and the exact frame names that support it. Do not confuse temporal succession with a causal relationship. If a category contains no elements, return an empty list. Return exclusively a valid JSON object."*

```
    Non confondere una successione
    temporale con una relazione causale.
    Se una categoria non contiene elementi,
    restituisci una lista vuota.
    Restituisci esclusivamente
    un oggetto JSON valido."""
```

Models are constrained to output structured JSON objects containing standardized metadata including `start_time`, `end_time`, `evidence_frames`, `evidence_type`, and `confidence`. Entity identifiers generated within a window can be cross-referenced across actions, events, and relations in the same segment. The main execution loop is encapsulated in `process_video_with_inf`. After dense frames are partitioned into temporal windows, each window is processed sequentially:

```
for window_index, window in enumerate(
    windows,
    start=1,
):
    prompt = build_semantic_prompt(
        video_directory.name,
        window,
    )
    response = inferencer(
        window.frame_paths,
        prompt,
    )

    parsed = parse_model_json(response)

    segment = normalize_semantic_output(
        parsed,
        window,
    )
```

```
20
21      segments.append(segment)
```

The `inferencer` component represents the sole model-dependent step in this loop, accepting frame paths and prompts to return textual outputs. All downstream parsing and validation operations remain identical across architectures. To manage output formatting inconsistencies from generative models, the parsing stage strips Markdown wrappers and extracts the primary JSON object. If standard decoding fails, the pipeline attempts recovery using `json_repair` before marking the response invalid:

```
try:
    parsed = json.loads(candidate)
except json.JSONDecodeError as first_error:
    try:
        from json_repair import repair_json
        parsed = json.loads(
            repair_json(candidate)
        )
    except Exception as repair_error:
        raise SemanticOutputError(
            f"JSON non valido: {first_error}"
        ) from repair_error
```

Structural constraints are subsequently enforced by `normalize_semantic_output`. This function clips temporal boundaries to window limits, restricts confidence scores to the interval $[0, 1]$, and filters out invalid frame references. Evidence types are validated against `observed`, `inferred`, or `uncertain`, automatically reclassifying causal hypotheses as `inferred` if initially marked as `observed`:

```
confidence = _safe_float(
    item.get("confidence"),
    0.0,
)
```

```
item["confidence"] = round(
    max(0.0, min(1.0, confidence)),
    4,
)
evidence_type = str(
    item.get(
        "evidence_type",
        "observed",
    )
).lower()
if evidence_type not in {
    "observed",
    "inferred",
    "uncertain",
}:
    evidence_type = "uncertain"
if (
    category == "causal_hypotheses"
    and evidence_type == "observed"
):
    evidence_type = "inferred"
```

Frame-level grounding validation retains only filenames present in the input temporal window:

```
valid_evidence = set(
    window.evidence_frames
)
item["evidence_frames"] = [
    str(name)
    for name in evidence_frames
    if str(name) in valid_evidence
]
```

Normalized segments are enriched with temporal metadata from the preprocessing stage:

```
normalized = {
    "segment_id": window.segment_id,
    "start_time": window.start_time,
    "end_time": window.end_time,
    "input_frames": window.evidence_frames,
}
```

utputs are saved per video in a dedicated `_semantic.json` file alongside model metadata and window configuration settings. Window failures are logged in an `errors` field containing segment identifiers, temporal bounds, frame lists, and error descriptions, preserving optional raw responses for diagnostic review. Ultimately, this phase transforms temporally indexed visual evidence into structured local representations. Cross-window deduplication is omitted at this step to preserve local grounding, leaving element consolidation and full video reconstruction to subsequent analysis stages.

Event Analysis

Spatial Relations

Spatial information captures entity configurations within the scene and tracks positional changes across localized visual observations. Unlike temporal relations, spatial features are extracted directly during the semantic representation phase while visual windows remain accessible. Spatial relations encode explicit configurations between entities or between an entity and scene elements. These encompass positional, proximity, containment, and orientation relations, including *left of*, *right of*, *above*, *below*, *inside*, *outside*, *in front of*, *behind*, *near*, and *on*. Because these descriptors tie to localized segments, a single entity pair may exhibit distinct spatial arrangements across different temporal intervals. Displacements and state transitions also drive spatial variations. A positional change combines spatial relations observed at different timestamps, corresponding actions, and recorded `state_changes`, thereby distinguishing static scene layouts from dynamic rearrangements. Furthermore, event-dependent localization evaluates spatial configurations

relative to an event interval (before, during, or after), isolating how previous actions alter an entity's final position. Methodologically, co-occurrence does not imply a spatial relation. The presence of two entities within the same temporal segment provides insufficient grounds to assume proximity, orientation, containment, or contact. A relation is retained only when explicitly captured in the semantic analysis. Currently, spatial evidence remains bound to local semantic descriptions. The event extraction stage leverages `spatial_relations` for event reconstruction without constructing a global spatial graph.

Temporal Relations

Temporal information is formalized during event extraction, where local semantic descriptions consolidate into a chronological timeline covering the full video. Each consolidated event retains a `start_time` and an `end_time` to define its temporal boundary and duration. Relations between events are explicitly modeled using pairs of `event_id` references. Supported relation types include *before*, *after*, *overlaps*, *during*, and *simultaneous*. Each relation links two existing events in the consolidated representation alongside an associated confidence score. The terms *before* and *after* define temporal precedence and sequence. *During* denotes temporal containment, *overlaps* indicates partial interval sharing, and *simultaneous* denotes concurrent occurrences. This structure captures chronological succession, repetition, containment, and overlap across the video timeline while preserving distinct occurrences. Crucially, generation order within the JSON output does not constitute temporal evidence. Dependencies derive strictly from segment timestamps and relational metadata rather than model output sequence. The temporal schema adopts the following structure:

```
"temporal_relations": [
{
"first_event": "E0001",
"relation": "before|after|overlaps|during|simultaneous",
"second_event": "E0002",
"confidence": 0.0
}
```

```
]
```

This consolidated event structure serves as input for causal analysis. Chronological organization is kept distinct from causal inference, as temporal precedence does not establish a cause and effect relationship.

Implementation Details The event analysis is implemented as a second-stage transformation over the structured semantic representations produced by the previous phase. No visual input is processed again at this stage. For each model, the corresponding semantic output is loaded and used as the only source of information for event reconstruction. The event analyzer can reorganize and consolidate previously extracted information, but it cannot introduce evidence obtained from a new inspection of the video. This preserves the separation between perceptual extraction and subsequent structural reasoning. The input therefore consists of the temporally indexed semantic segments generated for the same model. Each segment retains its identifier and temporal boundaries together with the semantic categories extracted from the corresponding visual window. The information made available to the event analysis includes entities, actions, local events, spatial relations, state changes, temporal relations, and causal hypotheses:

```
keys = (
"segment_id",
"start_time",
"end_time",
"entities",
"actions",
"events",
"spatial_relations",
"state_changes",
"temporal_relations",
"causal_hypotheses",
)
```

These fields are jointly considered because an event may not be represented exclusively

in the local `events` field. For instance, an action or an observable state transition may provide relevant information for reconstructing an event even when the local semantic description does not encode it as an independent event. The event analysis consequently operates over the complete local semantic representation rather than simply concatenating the previously extracted event lists. Before inference, the semantic segments are arranged according to their temporal position and serialized into a structured textual input. The model is explicitly instructed to treat this representation as a closed source of evidence. The common prompt constrains the reconstruction process through a set of rules designed to prevent additional perceptual or commonsense information from being introduced:

```

* usa esclusivamente le informazioni presenti
nell'analisi semantica
* non inventare azioni, oggetti, intenzioni,
emozioni o cause mancanti
* unisci soltanto i duplicati causati dalla
sovrapposizione delle finestre
* mantieni separati eventi realmente distinti
o ripetuti
* ricava i tempi soltanto dai segmenti forniti
```

A central function of this phase is the consolidation of local descriptions produced by overlapping temporal windows. Since the semantic extraction uses 4-second windows with a 3-second stride, adjacent segments share part of the same visual evidence, and the same physical event can consequently appear in multiple semantic outputs. During event reconstruction, descriptions referring to the same occurrence are merged into a single event. The operation is intended to remove redundancy introduced by the preprocessing strategy rather than repetitions that genuinely occur in the video. The prompt explicitly enforces this distinction. Events with similar descriptions are not automatically treated as duplicates: two occurrences are merged only when their temporal context and supporting segments indicate that they represent the same event observed through overlapping windows. Repeated actions occurring at different moments, by contrast, remain represented as dis-

tinct events, so that repetition is preserved as a meaningful temporal property of the video. The consolidated representation follows a common JSON schema. Each reconstructed event contains a unique identifier, a concise description, temporal boundaries, the participants involved, the semantic segments supporting the event, the type of evidence, and a confidence value:

```
{
"event_id": "E0001",
"description": "string",
"start_time": 0.0,
"end_time": 0.0,
"participants": ["entity or participant"],
"evidence_segments": ["segment_0001"],
"evidence_type":
"observed|inferred|uncertain",
"confidence": 0.0
}
```

The `event_id` field provides a stable reference for subsequent relational analyses. Events are assigned progressive identifiers such as `E0001`, `E0002`, and `E0003`, associated with the consolidated chronological representation rather than inherited from the local semantic outputs, so that events can be referred to consistently after multiple local descriptions have been merged. Temporal localization is inherited from the semantic analysis. The event extractor does not estimate event boundaries by accessing the original frames. Instead, `start_time` and `end_time` must remain compatible with the temporal intervals of the semantic segments supporting the event. When the same occurrence is represented across multiple overlapping windows, its consolidated interval can reflect the temporal evidence distributed across those segments. The `evidence_segments` field preserves provenance after consolidation. An event extracted from a single semantic window may contain only one segment identifier, whereas an event observed across overlapping windows may be associated with multiple segments:

```
"evidence_segments": [
```

```
"segment_0004",
"segment_0005"
]
```

Consolidation removes duplicated event descriptions without discarding the evidence from which they originated, which is essential for keeping the resulting event representation traceable. The event representation is therefore more compact than the semantic representation while remaining traceable to the local observations produced in the previous phase. Participants are derived from entities or participants already represented in the semantic analysis and are not generated as independent new entities during this stage. The event representation maintains the semantic continuity between entity extraction and event reconstruction in this way. The epistemic status of each event is represented through `evidence_type`. The analysis distinguishes three values:

```
"observed"
"inferred"
"uncertain"
```

`Observed` is used for events supported by information previously represented as directly visible. `Inferred` is reserved for information that already requires an inferential interpretation of the available semantic evidence. `Uncertain` represents cases in which the available evidence does not support a stronger classification. The event extraction stage is not intended to transform directly observed information into unsupported inference. Confidence is represented numerically in the $[0, 1]$ interval and is subsequently normalized to ensure a consistent representation. Normalization also verifies the structural validity of the generated output before serialization. In addition to individual events, the same inference produces an initial global temporal organization. Temporal relations connect events through their newly assigned identifiers rather than through their textual descriptions:

```
"temporal_relations": [
{
"first_event": "E0001",
"relation":
```

```
"before|after|overlaps|during|simultaneous",
"second_event": "E0002",
"confidence": 0.0
}
]
```

The allowed relations distinguish precedence, succession, containment, partial temporal overlap, and simultaneity. Their inclusion at this stage provides an explicit representation of the temporal structure that emerges once local events have been consolidated. Temporal relations cannot be derived solely from the order in which events are written in the generated JSON. The prompt explicitly requires temporal organization to depend on the intervals and relations available in the semantic representation. The order in which the text is generated is not treated as temporal evidence, and temporal succession is likewise kept distinct from causal interpretation. The model output is parsed as structured JSON rather than retained as free-form text. Responses are cleaned from possible Markdown delimiters, and the JSON object is extracted before decoding. The same robust parsing strategy used in the semantic stage allows malformed but recoverable outputs to be repaired when possible:

```
text = (
text.replace("‘json", "")
        .replace("‘", "")
.strip()
)

start = text.find("{")
end = text.rfind("}")

if start < 0 or end < start:
raise ValueError(
"La risposta non contiene un oggetto JSON"
)
```

```
14
15 payload = json.loads(
16 text[start:end + 1]
17 )
```

This phase primarily influences the granularity of representation. Semantic analysis captures evidence within specific, overlapping segments, whereas event analysis transforms these observations into a cohesive, event-centric depiction of the entire video. It eliminates redundancies from temporal overlap, retains authentically repeated instances, assigns consistent identifiers, preserves temporal and evidential origins, and sets up initial explicit connections among events, all without the need for additional visual input inspection.

Causal Inference

Causal inference operates exclusively on the consolidated event representation, functioning independently from perceptual extraction. This phase receives no visual frames or raw multimodal inputs, relying solely on previously reconstructed events and their established `temporal_relations`. Isolating causal reasoning prevents the recovery of omitted visual details or the insertion of unobserved events. Before processing, the event representation is pruned to essential fields: `event_id`, description, temporal interval, participants, supporting semantic segments, `evidence_type`, and confidence. The analysis evaluates four causal dependency types:

- `causes`: One event directly generates the outcome represented by another.
- `enables`: One event establishes the prerequisite conditions for a subsequent event.
- `motivates`: Visual evidence supports an intentional or motivational link.
- `prevents`: One event inhibits or counteracts another.

Each entry links a `cause_event` to an `effect_event`, optionally incorporating `supporting_events` to substantiate the inference. No new events are instantiated during this phase. The resulting structure follows this schema:

```
"causal_relations": [
  {
    "cause_event": "E0001",
    "relation": "causes|enables|motivates|prevents",
    "effect_event": "E0002",
    "evidence_type": "direct|inferred|uncertain",
    "supporting_events": ["E0003"],
    "explanation": "string",
    "confidence": 0.0
  }
]
```

The epistemic status of each relation is indicated by `evidence_type`. Dependencies grounded in explicit physical interactions or state transitions use `direct`. Connections requiring contextual interpretation use `inferred`, while partially supported links receive `uncertain`. Motivational dependencies require explicit support within the event representation rather than speculative plausibility. Temporal succession does not imply causation. A chronological link such as *before* or *after* remains insufficient to establish a causal relation. Dependencies are introduced only when the semantic content explicitly supports causation, enabling, motivation, or prevention. Separating these stages ensures temporal ordering is not mistaken for causal evidence, preserving the distinction between extracted facts and inferential reasoning.

Implementation Details The causal inference stage operates as a text-only processing step over the consolidated event representation produced in the preceding phase. For each video, the pipeline loads the corresponding `_events.json` file generated by the model. Crucially, no raw video frames, audio signals, transcriptions, or downstream text prompts are supplied. The causal analyzer thus operates exclusively on a closed representation of previously extracted visual information. Prior to inference, the event file is pruned via the `compact` function to retain only essential fields:

```
def compact(payload):
    return {
        "id_video": payload.get("id_video"),
        "events": [
            {
                "event_id": event.get("event_id"),
                "description": event.get("description"),
                "start_time": event.get("start_time"),
                "end_time": event.get("end_time"),
                "participants": event.get("participants", []),
                "evidence_segments": event.get("evidence_segments", [])
    ,
                "evidence_type": event.get("evidence_type"),
                "confidence": event.get("confidence"),
            }
            for event in payload.get("events", [])
        ],
        "temporal_relations": payload.get("temporal_relations", []),
    }
```

This filtering isolates causal reasoning from original perceptual features, requiring the model to derive dependencies strictly from consolidated events and their temporal layout. The implementation explicitly bounds the allowable relation types and evidence categories:

```
ALLOWED_RELATIONS = {
    "causes",
    "enables",
    "motivates",
    "prevents",
}
ALLOWED_EVIDENCE = {
    "direct",
```

```
    "inferred",
    "uncertain",
}
```

These definitions differentiate direct cause-and-effect links (`causes`) from enabling conditions (`enables`), intentional drivers (`motivates`), or inhibitory actions (`prevents`). Inference is guided by a standardized prompt instructing the model to rely solely on the provided events and temporal relations. The prompt explicitly forbids treating simple chronological succession as causal evidence:

```

usa esclusivamente gli eventi e le relazioni
temporali presenti nell'input;
non usare domande, caption, foil o altre
informazioni esterne;
la semplice successione temporale NON implica
causalita;
crea una relazione soltanto quando il contenuto
degli eventi supporta una dipendenza causa-effetto,
una condizione abilitante, una motivazione oppure
una prevenzione;
cause_event ed effect_event devono essere ID
di eventi presenti nell'input;
non inventare eventi, oggetti, intenzioni o cause
non ricavabili dalla rappresentazione fornita;
se non esiste evidenza sufficiente, restituisci
causal_relations come lista vuota;
```

Relationships are classified by epistemic confidence: `direct` for explicit physical or state-change interactions, `inferred` for contextual or intentional deductions, and `uncertain` for weakly supported cues. Output formatting enforces a structured JSON schema:

```
{
"causal_relations": [
```

```
{
    "cause_event": "E0001",
    "relation": "causes|enables|motivates|prevents",
    "effect_event": "E0002",
    "evidence_type": "direct|inferred|uncertain",
    "supporting_events": ["E0003"],
    "explanation": "string",
    "confidence": 0.0
}
]
}
```

The fields `cause_event` and `effect_event` reference existing event IDs, while `supporting_events` optionally logs contextually relevant secondary events. Output parsing strips Markdown delimiters and decodes the outer JSON object. If formatting errors occur, `json_repair` attempts structural recovery:

```
text = text.replace("```json", "").replace("```", "").strip()
start = text.find("{")
end = text.rfind("}")
candidate = text[start : end + 1]
try:
    return json.loads(candidate)
except json.JSONDecodeError:
    from json_repair import repair_json
    return json.loads(repair_json(candidate))
```

If parsing fails entirely, a secondary generation call requests strictly formatted JSON without altering input prompts.

A normalization step validates candidate relations against valid event IDs parsed from input:

```
valid_ids = {
    event.get("event_id")
```

```
    for event in payload.get("events", [])
    if event.get("event_id")
}
```

Self-referential links and invalid event IDs are rejected:

```
if (
    cause not in valid_ids
    or effect not in valid_ids
    or cause == effect
):
    continue
```

Unrecognized relation types are discarded, while invalid evidence labels default to `inferred`:

```
if rel not in ALLOWED_RELATIONS:
    continue
if evidence not in ALLOWED_EVIDENCE:
    evidence = "inferred"
```

Duplicate relations are deduplicated using unique (cause, relation, effect) triplets:

```
key = (cause, rel, effect)
if key in seen:
    continue
seen.add(key)
```

Auxiliary `supporting_events` entries are filtered to exclude invalid IDs or self-references:

```
supporting = [
    event_id
    for event_id in relation.get("supporting_events", [])
    if event_id in valid_ids and event_id not in {cause, effect}
]
```

Confidence scores are bounded within the $[0, 1]$ interval:

```
try:
    confidence = float(relation.get("confidence", 0.0))
except (TypeError, ValueError):
    confidence = 0.0
confidence = max(0.0, min(1.0, confidence))
```

Final outputs are serialized per video into `_causal.json` files containing metadata, model parameters, and target relations. Unresolvable errors yield empty relation arrays rather than speculative fallbacks.

Representation Integration and NLI-Based Visual Proficiency Assessment

The final stage of the preliminary analysis integrates the information produced at the semantic, event, temporal, spatial, and causal levels into a consolidated representation of the visual evidence recovered by each model. The procedure operates exclusively on the outputs generated during the preceding phases, preserving the separation between visual analysis and question-dependent evaluation without performing any further inspection of the video. The integrated representation combines complementary information produced at different levels of the pipeline. Entity descriptions, spatial relations, and state changes originate from the semantic analysis. Consolidated events provide a global event-level representation together with their participants, temporal boundaries, supporting segments, evidence type, and confidence. Temporal relations connect these events through their identifiers, while causal relations encode the dependencies established during the dedicated causal inference stage. These components are treated not as independent observations, but as progressively structured views of the same visual evidence. Finalization does not amount to a simple concatenation of all intermediate outputs. Redundancies introduced by overlapping temporal windows have already been resolved during event consolidation, where multiple descriptions of the same occurrence are merged while genuinely repeated events remain distinct, and duplicated or structurally invalid causal relations have

already been removed during causal normalization. The epistemic status assigned during the previous stages is preserved as well: information classified as `observed`, `inferred`, or `uncertain` remains distinguishable in the consolidated representation, together with the corresponding confidence values. Uncertain information is not automatically promoted to positive evidence, nor are conflicting descriptions resolved through an additional interpretation introduced at this stage. The final representation thus captures the entities, events, spatial configurations, temporal organization, and causal dependencies recovered by the model, without expanding the evidence beyond what the visual analysis actually produced. Crucially, **finalization does not constitute an answer to any VSV question**. Its purpose is to fix a representation of what the model has recovered from the visual modality before the linguistic content of the evaluation item is introduced. Questions, captions, and foils therefore play no role in constructing the representation, and once it has been finalized, its content remains fixed throughout the evaluation of the corresponding question: the question may provide context for interpreting the linguistic alternatives, but it cannot trigger the extraction of new entities, events, relations, or visual evidence. The proficiency assessment itself is formulated as a Natural Language Inference task. The finalized visual representation acts as the **premise**, while the caption and the corresponding foil are evaluated independently as alternative hypotheses. GPT-4o is employed exclusively as an NLI judge, not to answer the original question, generate a new description of the video, or select directly between the two alternatives. Its role is restricted to determining the logical relation between the evidence contained in the finalized representation and each hypothesis. For both caption and foil, the judge assigns one of three labels: `entailment`, `contradiction`, or `neutral`. `Entailment` indicates that the hypothesis is supported by the available visual representation. `Contradiction` indicates that it is incompatible with the represented evidence. `Neutral` is used when the available information is insufficient to establish either relation, so that missing evidence is not interpreted as contradiction and absence of information is not treated as negative evidence. The caption and foil are judged separately. A caption–foil pair receives `PASS` only when the finalized representation entails the caption and contradicts the foil. Every other combination produces `NOT PASS`. This rule adopts a deliberately conservative criterion. Entailment of the caption alone is

Caption	Foil	Outcome
Entailment	Contradiction	`PASS`
Any other combination		`NOT PASS`

Table 3.3: Decision rule for the NLI-based visual proficiency assessment.

insufficient, since the same representation must also provide enough evidence to reject the minimally contrastive foil. Contradiction of the foil, in turn, cannot compensate for a caption classified as `neutral`. A model is therefore considered proficient on a given pair only when its visual representation supports both sides of the required distinction. The procedure is repeated independently for the four caption–foil pairs associated with each question. Question-level proficiency requires consistency across the complete set: a question receives `PASS` only when all four pairs satisfy the pair-level criterion, and is classified as `NOT PASS` if at least one pair fails. This strict aggregation reduces the influence of individual linguistic formulations and requires the recovered visual evidence to remain sufficient across multiple, minimally different realizations of the same underlying item. For each pair, the procedure records the classification assigned to the caption, the classification assigned to the foil, and the resulting binary decision, which are then aggregated into the final proficiency label associated with the question. The resulting measure is accordingly distinct from conventional question-answering accuracy: rather than evaluating whether the model can generate or select the correct answer after being exposed to the question, it evaluates whether the information independently recovered from the visual channel is sufficiently complete and discriminative to support the correct statement while rejecting its minimally contrastive alternative, connecting question-independent visual analysis to the competence-oriented evaluation objective of the preliminary analysis.

Implementation Details The final evaluation is implemented in `final_evaluation.py`, which operates on the finalized representations produced for the four evaluated models. These representations are stored as JSON files in `data/preliminar_analysis/final_results` and constitute the fixed visual premises used throughout the proficiency assessment. The script expects one finalization file for each model and verifies their presence before starting

the evaluation:

```
EXPECTED_MODELS = (
"gemma",
"gemma-4-12B",
"qwen",
"qwen-30B",
)

FINAL_DIR = Path(
"data/preliminar_analysis/final_results"
)

final_files = [
FINAL_DIR / f"{model}.json"
for model in EXPECTED_MODELS
]
```

Finalized representations are loaded independently for each model and indexed by normalized video identifiers. The loading procedure accepts different top-level JSON organizations, including lists of video representations, dictionaries containing `videos` or `results`, individual video objects, and dictionaries directly indexed by video ID. This normalization allows the subsequent NLI procedure to retrieve the complete representation associated with a video without modifying its content. Question metadata and caption–foil pairs are loaded separately from `question_classification.csv` and `caption_foil.csv`. Video identifiers and question categories are normalized before the two sources are joined using the video and principal question dimension. The question is therefore introduced only after the finalized visual representation has been loaded, and is retained as contextual information for the NLI judge rather than incorporated into the premise itself. The NLI evaluation uses `gpt-4o-2024-08-06` with deterministic generation:

```
MODEL = "gpt-4o-2024-08-06"

```

```
risposta = client.chat.completions.parse(
model=MODEL,
temperature=0,
messages=[
{
"role": "system",
"content":
f"{PROMPT}\n\n{REGOLE[categoria]}",
},
{
"role": "user",
"content": contenuto,
},
],
response_format=RispostaNLI,
)
```

Each request contains three distinct fields: the question, the finalized representation used as the premise, and one hypothesis. The caption and foil are never evaluated jointly. Instead, two independent calls are performed using the same premise and question context:

```
nli_caption = classifica(
premessa,
row["domanda"],
row["categoria"],
row["caption"],
)

nli_foil = classifica(
premessa,
row["domanda"],
row["categoria"],
row["foil"],
```

```
)
```

The system prompt explicitly restricts the judge to the information contained in the premise. External knowledge, completion of missing information, and plausibility-based reasoning are excluded. Incomplete, weak, or ambiguous evidence must be classified as `neutro`. Additional constraints are introduced according to the question category: spatial relations cannot be inferred from simple co-occurrence, temporal relations cannot be inferred from the textual order of events, and causal relations cannot be established from temporal succession alone. The model response is constrained through a Pydantic schema rather than parsed from unrestricted natural-language output:

```
class RispostaNLI(BaseModel):
etichetta: Literal[
"implicazione",
"contraddizione",
"neutro",
]
```

This design restricts each inference to one of the three admissible NLI labels, preventing the judge from generating explanations or free-form answers. A missing or structurally invalid classification raises an error rather than being converted into an evaluation decision. Pair-level proficiency is subsequently computed through a deterministic rule implemented outside GPT-4o:

```
def valuta_coppia(caption, foil):
return (
"PASS"
if caption == "implicazione"
and foil == "contraddizione"
else "NOT PASS"
)
```

GPT-4o's role is thus limited to assigning the two NLI labels, and it does not directly decide whether a proficiency test is passed. The binary outcome is derived program-

matically, receiving PASS only when the finalized representation entails the caption and contradicts the corresponding foil. All other label combinations result in NOT PASS. Results are written incrementally to a model-specific checkpoint in the nli subdirectory. Before evaluating a pair, the script checks whether its identifier has already been processed, allowing interrupted executions to resume without repeating completed API calls. Each stored row contains the model, pair identifier, video, category, question, caption, foil, the two NLI classifications, and the resulting pair-level outcome. Once all model-specific evaluations have been completed, pair-level records are concatenated into nli_risultati.csv. Question-level proficiency is then computed by grouping results according to model, video, category, and question:

```
df = risultati.groupby(
[
"modello",
"video_id",
"categoria",
"domanda",
]
).agg(
totale_coppie=("esito", "size"),
coppie_pass=("pass", "sum"),
).reset_index()

df["proficiency"] = [
"PASS" if passate == totale else "NOT PASS"
for passate, totale in zip(
df["coppie_pass"],
df["totale_coppie"],
)
]
```

The aggregation adopts the same conservative principle used at pair level: proficiency is

assigned only when every caption–foil pair associated with the question is passed. Since the experimental data contain four pairs for each question, this corresponds to the required $4/4$ criterion, and a single `NOT PASS` is sufficient to classify the question as `NOT PASS`. The script additionally stores the number of evaluated pairs, the number of successful pairs, and their ratio in `question_proficiency.csv`. The implementation keeps the responsibilities of the two components distinct: the finalized JSON representation remains unchanged and functions solely as the visual premise, GPT-4o performs constrained NLI classification over each caption and foil, and the final proficiency decision is produced by deterministic aggregation rules rather than by an additional generative judgment.

4. Results and Discussions

4.1 Preliminar Analysis Result

Expectation

5. Conclusions

Conclusioni...

6. Future Perspectives

List of Abbreviations

VLM	Visual Language Model
VQA	Visual Question Answering
AGQA	Action Genome Question Answering
MVBench	Multi-modal Video Understanding Benchmark
VILMA	Video Language Model Assessment
MAIA	Multimodal AI Assessment
VSV	Visual Statement Verification
OEVQA	Open-Ended Visual Question Answering
MLM	Multimodal Language Model
AVQA	Audio-Visual Question Answering
Dave	Diagnostic benchmark for Audio Visual Evaluation
MMAU-Pro	Massive Multi-Task Audio Understanding and Reasoning benchmark, Pro version
SONIC-01	SOcial Natural Interaction Corpus (Omnimodal, v1)
GPT	Generative Pre-trained Transformer
MIRAGE	Assessing Hallucination in Multimodal Reasoning Chains of Multimodal Large Language Models

Bibliography

[1] Stanislaw Antol et al. “VQA: Visual Question Answering”. In: *Proceedings of the IEEE International Conference on Computer Vision*. 2015, pp. 2425–2433.

[2] Marco Baroni. “Grounding Distributional Semantics in the Visual World”. In: *Language and Linguistics Compass* 10 (2015). DOI: 10.1111/lnc3.12170.

[3] Zesen Cheng et al. “VideoLLaMA 2: Advancing Spatial-Temporal Modeling and Audio Understanding in Video-LLMs”. In: *arXiv preprint arXiv:2406.07476* (2024). arXiv: 2406.07476. URL: https://arxiv.org/abs/2406.07476.

[4] Michela Deodati et al. “Lost in Transcription: Investigating the Role of Audio Information for Video-Based Visual Statement Verification”. In: *Proceedings of the Twelfth Italian Conference on Computational Linguistics (CLiC-it 2026)*. Forthcoming. 2026.

[5] Bowen Dong et al. “MIRAGE: Assessing Hallucination in Multimodal Reasoning Chains of MLLMs”. In: *Advances in Neural Information Processing Systems*. 2025. URL: https://bit.ly/25mirage.

[6] Gemma Team. *Gemma 4 Technical Report*. 2026. DOI: 10.48550/arXiv.2607.02770. arXiv: 2607.02770 [cs.CL]. URL: https://arxiv.org/abs/2607.02770.

[7] Rohit Girdhar et al. “ImageBind: One Embedding Space to Bind Them All”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 15180–15190.

[8] Madeleine Grunde-McLaughlin, Ranjay Krishna, and Maneesh Agrawala. “AGQA: A Benchmark for Compositional Spatio-Temporal Reasoning”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, pp. 11287–11297.

[9] Drew A. Hudson and Christopher D. Manning. “GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 6700–6709.

[10] Ilker Kesen et al. “VILMA: A Zero-Shot Benchmark for Linguistic and Temporal Grounding in Video-Language Models”. In: *International Conference on Learning Representations*. 2024. arXiv: 2311.07022. URL: https://arxiv.org/abs/2311.07022.

[11] Sonal Kumar et al. “MMAU-Pro: A Challenging and Comprehensive Benchmark for Holistic Evaluation of Audio General Intelligence”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 2026, pp. 22688–22697. URL: https://sonalkum.github.io/mmau-pro.

[12] Guangyao Li et al. “Learning to Answer Questions in Dynamic Audio-Visual Scenarios”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 19108–19118.

[13] K. Li et al. “MVBench: A Comprehensive Multi-Modal Video Understanding Benchmark”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024. DOI: 10.48550/arXiv.2311.17005. arXiv: 2311.17005.

[14] Qwen Team. “Qwen2.5-Omni Technical Report”. In: *arXiv preprint arXiv:2503.20215* (2025). arXiv: 2503.20215. URL: https://arxiv.org/abs/2503.20215.

[15] Qwen Team. “Qwen3.5-Omni Technical Report”. In: *arXiv preprint arXiv:2604.15804* (2026). arXiv: 2604.15804. URL: https://arxiv.org/abs/2604.15804.

[16] Gorjan Radevski et al. “Dave: Diagnostic benchmark for audio visual evaluation”. In: *Advances in Neural Information Processing Systems* 38 (2026).

[17] Ahmed Y. Radwan et al. “SONIC-O1: A Real-World Benchmark for Evaluating Multimodal Large Language Models on Audio-Video Understanding”. In: *arXiv preprint arXiv:2601.21666* (2026). arXiv: 2601.21666. URL: https://arxiv.org/abs/2601.21666.

[18] Omar Sanseviero and Ian Ballantyne. *Introducing Gemma 3n: The Developer Guide*. Google Developers Blog. June 2025. URL: https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/.

[19] Ravi Shekhar et al. “FOIL it! Find One Mismatch between Image and Language Caption”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2017, pp. 255–265. DOI: 10.18653/v1/P17-1024. URL: https://aclanthology.org/P17-1024/.

[20] Guangzhi Sun et al. “Video-SALMONN: Speech-Enhanced Audio-Visual Large Language Models”. In: *arXiv preprint arXiv:2406.15704* (2024). arXiv: 2406.15704. URL: https://arxiv.org/abs/2406.15704.

[21] Davide Testa et al. “All-in-One: Understanding and Generation in Multimodal Reasoning with the MAIA Benchmark”. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. 2025, pp. 20030–20050. DOI: 10.18653/v1/2025.findings-emnlp.1091. URL: https://aclanthology.org/2025.findings-emnlp.1091/.

[22] Davide Testa et al. “MAIA: A Benchmark for Multimodal AI Assessment”. In: *Proceedings of the Eleventh Italian Conference on Computational Linguistics (CLiC-it 2025)*. 2025, pp. 1121–1134. URL: https://aclanthology.org/2025.clicit-1.106/.

[23] Tristan Thrush et al. “Winoground: Probing Vision and Language Models for Visio-Linguistic Compositionality”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 5238–5248.

[24] Jin Xu et al. “Qwen2.5-Omni Technical Report”. In: *arXiv preprint arXiv:2503.20215* (2025). DOI: 10.48550/arXiv.2503.20215. URL: https://arxiv.org/abs/2503.20215.

[25] Jin Xu et al. “Qwen3-Omni Technical Report”. In: *arXiv preprint arXiv:2509.17765* (2025). DOI: 10.48550/arXiv.2509.17765. URL: https://arxiv.org/abs/2509.17765.

Appendix

A. Appendix